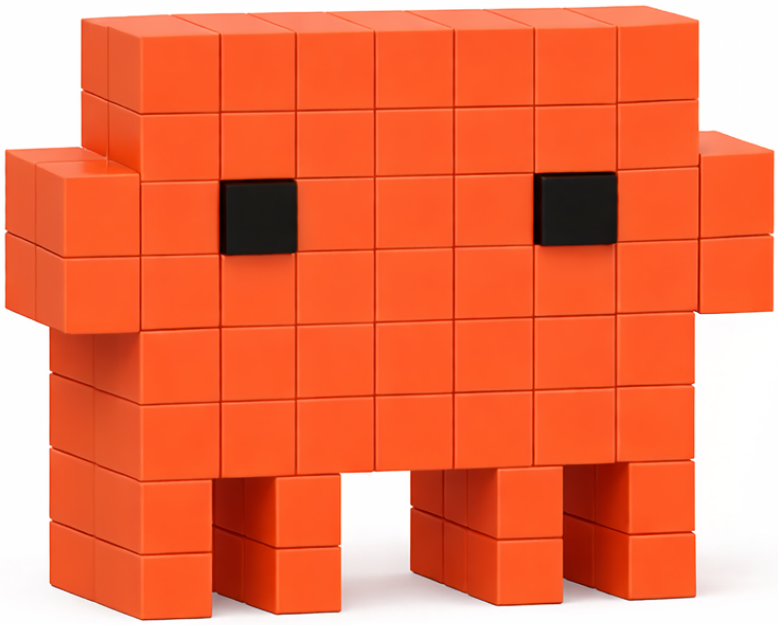
# CLAUDE CODE FOR DEVELOPER EXPERIENCE



ARTEM MUKHIN ▪ DX-RAY.COM

Предисловие
- Для кого
- Как читать
- О примерах
- Об авторе

60 минут до первого результата
- Минута 0-5: Установка
- Минута 5-15: CLAUDE.md
- Минута 15-25: Первый skill
- Минута 25-35: Permissions
- Минута 35-60: Первый DX-кейс
- Что дальше

Глава 1: Зачем это DX-инженеру
- Что это вообще такое
- “Я боюсь консоли”
- Три модели использования
- Чем отличается от Copilot и Cursor
- Главная идея этой книги
- Bottom-Up DevEx

Глава 2: Установка и первые шаги
- Установка
- Pricing
- Первый запуск
- /init - стартовый CLAUDE.md
- Thinking modes
- Plan Mode - исследование без страха
- Checkpoints - эксперименты без риска
- Session management
- Первый полезный результат за 10 минут

Глава 3: CLAUDE.md - конституция проекта
- Что это такое
- Иерархия
- Что включать
- Что НЕ включать
- Лимит ~150 строк
- @ imports

- /init - старт с нуля
- Emphasis для важного
- CLAUDE.md для DX-инженера
- advisory vs deterministic
- Карта платформы

Глава 4: Skills - переиспользуемые workflows

- Формат SKILL.md
- Ключевые параметры
- Где хранить
- Когда skill, а когда скрипт
- Пример: skills для ежедневной разработки
- Пример: от бага до деплоя
- Skills для команды
- Чеклист

Глава 5: Hooks - детерминированная автоматизация

- Принцип: код > AI
- Как устроены hooks
- 3 core рецепта
- Комбинирование hooks
- Hooks для команды
- За пределами basics

Глава 6: MCP - подключение к внешнему миру

- Как работает
- Must-have серверы
- Другие полезные серверы
- Scopes
- SDK/CLI > MCP
- MCP + Skills = сильная комбинация
- Tool Search

Глава 7: Продвинутая кастомизация

- Subagents: изолированные контексты
- Worktrees: git-изоляция
- Auto-memory
- Non-interactive mode
- Sandboxing и Permissions
- Plugins
- Agent Teams и Remote Control

- Лестница расширения

Глава 8: Парадокс AI-продуктивности

- METR: AI замедляет опытных разработчиков
- Индивидуально лучше, организационно - никак
- Когда AI реально помогает
- Когда AI мешает
- 7 AI-amplifying capabilities
- Bottom-Up DevEx: другой подход к внедрению
- Личный опыт: от paralysis к shipping

Глава 9: Погружение в проект за час

- Кейс: 4 месяца за час
- Just-in-time learning
- CLAUDE.md как документ для новичка
- Plan Mode для первого погружения
- Ограничения и workarounds
- 5 файлов для DX-инфраструктуры

Глава 10: Документация и Knowledge Management

- Почему markdown
- Frontmatter caching
- Authorship tracking
- Структура документации для AI
- Filesystem > RAG
- Claude Code для документации

Глава 11: Code Review, Quality и Safety

- Writer/Reviewer pattern
- @claude в GitHub PR
- Verification = #1 practice
- Claude Code Security
- Non-negotiables
- Чего избегать

Глава 12: Claude Code в команде

- Shared CLAUDE.md (проверено)
- Shared Skills (проверено)
- Shared Hooks (проверено)
- GitHub Actions (enterprise)
- Permission Templates (проверено)
- Adoption Playbook

- Когда Claude Code НЕ нужен
- Anti-patterns
- Enterprise данные
- Claude Code по ролям

Глава 13: Как измерить DX-улучшения

- DX Core 4
- SPACE Framework
- DevEx (ACM Queue)
- DORA
- Mapping: Claude Code → DX метрики
- Практический подход
- Чего избегать

Заключение: Bottom-Up DevEx

- Bottom-Up DevEx: 5 принципов
- Иерархия автоматизации
- Четыре компонента платформы
- Что делать с понедельника
- Честное закрытие
- Дальше

Приложения

- Приложение A: Шаблоны
- Приложение B: Чеклист внедрения
- Приложение C: Ссылки и ресурсы
- Приложение D: Bottom-Up DevEx Checklist
- Приложение E: Когда НЕ использовать Claude Code
- Приложение F: Дополнительные рецепты hooks
- Приложение G: DX Survey Frameworks

# Предисловие

Гайдов по Claude Code десятки. Большинство про одно и то же: установи, напиши промпт, смотри как AI генерирует код. Полезно на первый час. Потом начинаются вопросы, на которые tutorials не отвечают: как встроить это в существующие процессы? Как измерить, что стало лучше, а не просто “ощущение такое”? Что делать, когда данные показывают, что AI замедляет опытных разработчиков на 19%?

Эта книга - не tutorial. Это DX engineering book: как использовать Claude Code для улучшения developer experience - своего и своей команды. С данными, с ограничениями, с честным разбором когда AI помогает, а когда мешает.

## Для кого

- **DX-инженеры** - ваша прямая задача: снижать friction для разработчиков. Здесь - конкретные инструменты для этого
- **Tech leads** - code review, качество, стандарты команды. Как не утонуть в PR, которые стали в 2.5 раза больше
- **Platform engineers** - автоматизация, CI/CD интеграция, batch-обработка
- **Любой инженер**, который хочет внедрить AI coding tools осмысленно, а не “поставил и разберусь”

Кому книга не подойдёт: тем, кто ищет список промптов или пошаговый tutorial для начинающих. Quick Win в начале даёт быстрый старт, но дальше - про процессы, измерения и архитектуру автоматизации.

## Как читать

Две части. **Часть 1** (главы 1-7): Claude Code как платформа - от установки до продвинутой кастомизации. **Часть 2** (главы 8-13): применение для DX - onboarding, документация, code review, измерение результатов.

Можно читать по порядку. Можно начать с главы 8 (парадокс AI-продуктивности), если интересует “зачем” прежде чем “как”. Каждая глава начинается с TL;DR - хватает для решения “читать ли целиком”.

## О примерах

Примеры написаны на macOS. На Linux и Windows всё работает аналогично - адаптация минимальна (пути, команды уведомлений).

Книга отражает состояние на март 2026. Claude Code обновляется еженедельно - актуальные фичи и цены на anthropic.com/claude-code.

## Об авторе

Artem Mukhin. 10+ лет в software engineering, специализация - Developer Experience. В Microsoft через серию DX-инициатив добился создания full-time DX engineering роли в Skype и проработал в ней год. Сейчас - в Teams. Создатель DX-Ray - knowledge hub и community по developer experience. Фреймворк Bottom-Up DevEx, описанный в этой книге, вырос из практики и опубликован в The New Stack. Больше о проектах и публикациях - artem-mukhin.com.

Эта книга написана с помощью инструментов, которые в ней описаны. CLAUDE.md задаёт структуру, hooks проверяют форматирование, Plan Mode использовался для исследований. Процесс написания - сам по себе тест описанного подхода.

# 60 минут до первого результата

Когда я впервые установил Claude Code, я получил первый полезный результат через 10 минут. Не “привет мир”, а реальное описание архитектуры проекта, в котором разбирался неделю. Вот тот же путь, но структурированный.

## Минута 0-5: Установка

```
curl -fsSL https://claude.ai/install.sh | bash
cd ~/your-project
claude
```

Авторизуйся через браузер. Готово.

## Минута 5-15: CLAUDE.md

```
/init
```

Claude проанализирует проект и создаст стартовый CLAUDE.md. Открой, отредактируй: убери лишнее, добавь gotchas которые знаешь только ты. Подробности в главе 3.

## Минута 15-25: Первый skill

Создай `.claude/skills/explain/SKILL.md`:

```
---
name: explain
description: Explain a part of the codebase
---
1. Find the relevant files for $ARGUMENTS
2. Read them carefully
3. Explain: what it does, how data flows, key abstractions
4. Note any gotchas or non-obvious behavior
```

Теперь `/explain authentication flow` - и Claude разберёт любой кусок кода. Глава 4.

## Минута 25-35: Permissions

Добавь в `.claude/settings.json`:

```
{
  "permissions": {
    "allow": [
      "WebFetch", "WebSearch",
      "Bash(ls:*)", "Bash(cat:*)", "Bash(git status:*)",
      "Bash(git log:*)", "Bash(git diff:*)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)"
    ]
  }
}
```

`allow` - безопасные команды без переспроса (иначе Claude спрашивает на каждый `ls`). `deny` - опасные команды заблокированы полностью. Глава 5.

## Минута 35-60: Первый DX-кейс

```
Прочитай README, CLAUDE.md и основные модули проекта.
Найди 5 главных источников friction для нового разработчика:
- что непонятно из документации
- какие gotchas не задокументированы
- где setup может сломаться
Для каждого предложи конкретное улучшение.
```

Результат: список конкретных DX-улучшений за 25 минут вместо недели наблюдений.

## Что дальше

За 60 минут: CLAUDE.md (контекст), skill (workflow), permissions (safety) и первый DX-результат. Это 80% ежедневного использования.

Остальные 13 глав - как масштабировать, измерить эффект и избежать типичных ошибок. Читай по мере необходимости.

# Глава 1: Зачем это DX-инженеру

> **TL;DR:** Claude Code - не автокомплитер, а агентная среда для автоматизации DX-процессов. Не “AI пишет код за тебя”, а “AI убирает friction, который мешает тебе делать свою работу”. 80% автоматизации - детерминированные (hooks, скрипты), 20% - AI.

---

За первый месяц с Claude Code я сделал в личных проектах больше, чем за предыдущие несколько лет. Не потому что стал работать по 16 часов. А потому что убрал friction, который раньше убивал всё между “придумал” и “закоммитил”.

Инструмент, который это сделал - Claude Code. Но эта книга не про “смотрите какой классный AI”. Она про то, как DX-инженер может использовать его для улучшения developer experience - своего и своей команды.

Ключевое отличие от ChatGPT, Copilot Chat и подобных: Claude Code - execution partner, не зеркало для рефлексии. Разница - в артефактах. Не “обсудили и закрыли вкладку”, а git-коммиты, shipped продукты, работающие автоматизации.

## Что это вообще такое

Claude Code - terminal-native agentic environment от Anthropic. Три слова, каждое важно:

**Terminal-native.** Работает в терминале. Не плагин к IDE, не веб-чат, не автокомплитер. Ты пишешь промпт - Claude читает файлы, запускает команды, вносит изменения, проверяет результат. Всё в том же окне, где ты и так работаешь.

**Agentic.** Не отвечает на вопросы - выполняет задачи. “Прочитай epic в issue tracker’е, все подтикеты, все PR, покажи summary” - и он идёт, читает, структурирует, отдаёт результат. Не одна подсказка, а цепочка действий.

**Environment.** Не одноразовый инструмент, а среда. С конфигурацией (CLAUDE.md), автоматизацией (hooks), расширениями (skills, MCP, plugins), памятью между сессиями. Это платформа, которую настраиваешь под себя.

## “Я боюсь консоли”

Если ты это подумал - ничего страшного, серьёзно. Но современный TUI - это не bash 1990-х.

Claude Code - это rich interface прямо в терминале. Подсветка кода, визуальные диффы, навигация, Plan Mode для исследования без изменений, checkpoints с откатом. Если ты хоть раз набирал `git status` - тебе хватит.

А если всё-таки хочется GUI - есть VS Code extension, где Claude Code работает в sidebar с inline-диффами и кнопками. Но терминальная версия мощнее, потому что скриптуема: hooks, non-interactive mode, пайплайны. Для DX-инженера это принципиально - ты же строишь автоматизации, а не просто код пишешь.

## Три модели использования

**Быстрый вопрос.** “Как работает X в этом проекте?” - Claude ищет по файлам и объясняет. Замена Stack Overflow, но с контекстом твоего кода.

**Задача.** “Добавь frontmatter ко всем markdown файлам в папке” - Claude пишет скрипт, запускает, проверяет результат. Замена 30-60 минут рутинной работы.

**Автономная работа.** “Прочитай этот epic, все связанные PR, выстрой pipeline и сделай HTML-отчёт” - Claude работает минуты, ты занимаешься другим. Замена часов или дней ручного погружения.

Для DX-инженера третья модель - самая ценная. Потому что DX-работа - это часто не “написать код”, а “разобраться в 4-месячном проекте”, “проанализировать тысячи сообщений из Slack-каналов команды”, “сгенерировать отчёт по всем репозиториям”.

## Чем отличается от Copilot и Cursor

Важный контекст: к 2026 году все три инструмента обзавелись agent-режимами. Copilot уже не “просто автокомплит”, Cursor - не “просто чат в IDE”. Разница теперь не в наличии агента, а в архитектуре и философии.

- **GitHub Copilot** - экосистема GitHub. Лучший inline autocomplete + Coding Agent, который берёт issue и создаёт PR прямо в GitHub Actions. Skills, MCP, plugins, custom agents. Сила - в интеграции: Issues → Agent → PR → Code Review - всё внутри одной платформы

- **Cursor** - AI-native IDE (форк VS Code). Agent Mode в редакторе + Background Agents в облачных VM (до 8 параллельно в Cursor 2.0). MCP, rules files. Сила - AI встроен в сам редактор, не прикручен сбоку
- **Claude Code** - terminal-native agent. Работает там, где живут скрипты и пайплайны. Hooks, skills, MCP, plugins. Сила - скриптуемость: всё конфигурируется кодом (CLAUDE.md, hooks, skills), встраивается в CI/CD, автоматизации

Это не “один лучше другого” - это разные точки входа. Многие разработчики используют 2-3 инструмента параллельно.

| | **Copilot** | **Cursor** | **Claude Code** |
|---|---|---|---|
| Среда | IDE-плагин + GitHub | AI-native IDE | Терминал |
| Agent-режим | Coding Agent (из issues, в GitHub Actions) | Background Agents (облачные VM) | Agentic execution (локально, скриптуемо) |
| Расширяемость | MCP, skills, plugins, custom agents | MCP, rules files | Hooks, skills, MCP, plugins |
| Суперсила | GitHub-экосистема от issue до deploy | IDE-first UX, параллельные агенты | Скриптуемость, composability, CI/CD |
| DX-применение | Код быстрее + PR-автоматизация | Код с контекстом + multi-agent | Процессы, автоматизации, onboarding |

На практике я использую оба: Claude Code для DX-задач (автоматизации, research, onboarding), Copilot в фоне для автокомплита в IDE. Они не конкурируют - они дополняют.

## Главная идея этой книги

Существующие гайды по Claude Code фокусируются на “пиши код быстрее”. Это полезно, но не про DX.

Developer Experience - это не скорость кодинга. Это friction на всём пути: от “не понимаю проект” до “задеплоил и забыл”. Onboarding, документация, code review, knowledge management, процессы в команде.

Claude Code для DX-инженера - не “быстрее кодить”. Это optimizer: берёт существующие процессы и убирает из них friction. Не создаёт с нуля - усиливает то, что уже работает:

- **CLAUDE.md** - конституция проекта, которая работает для каждой сессии и каждого нового разработчика
- **Hooks** - детерминированная автоматизация: форматирование, проверки, уведомления. Гарантированно, а не “надеюсь Claude послушается”
- **Skills** - повторяемые workflows одной командой: `/deploy`, `/fix-issue 123`, `/onboard`
- **MCP** - подключение к внешним сервисам (Notion, Chrome DevTools), когда нативного CLI/SDK недостаточно

И ключевой принцип, который мы будем развивать дальше:

> Всё, что можно сделать кодом - делай кодом. AI вызывается только там, где нужно творчество или рассуждение. 80% автоматизации должны быть детерминированными. 20% - AI.

Это контринтуитивно для гайда про AI-инструмент. Но именно это делает систему надёжной, быстрой и дешёвой.

## Bottom-Up DevEx

Есть два подхода к внедрению AI в DX.

**Top-down:** “Купили 500 лицензий, разберётесь. Вот enterprise platform.” Индустриальные данные показывают, что это не работает - delivery metrics остаются flat даже при кратном росте merged PR.

**Bottom-up:** Один инженер. Один CLAUDE.md. Один hook. Измерил результат. Расширил на команду. Не покупка платформы, а рост системы из практики.

Эта книга - про второй подход. Каждая глава - конкретный шаг, который можно сделать в одиночку, без бюджета и без разрешения. Результаты масштабируются, когда доказаны на практике.

---

# Глава 2: Установка и первые шаги

> **TL;DR:** Три команды до первого результата. Plan Mode для безопасного исследования, checkpoints для смелых экспериментов, /clear между задачами.

---

Я долго не переходил на agent CLI. Не потому что боялся консоли - я уже пользовался Claude через Copilot Chat в VS Code: он читал файлы, предлагал изменения, отвечал на вопросы. Казалось, что это и есть “AI в разработке”.

Я сильно ошибался. Agent CLI - это на порядки круче чата в IDE. У тебя доступ ко всем консольным утилитам, можно написать скрипт и тут же запустить, подключить любой API, автоматизировать целые workflows. Бесконечные возможности - и всё это в одном агентном цикле, где Claude сам решает что делать дальше.

Через 10 минут после установки я получил первый полезный результат. Через час - уже не понимал, как работал без этого.

## Установка

```
# 1. Установить
curl -fsSL https://claude.ai/install.sh | bash

# 2. Запустить в папке проекта
cd ~/my-project
claude
```

Требуется Node.js 18+. На macOS также работает `brew install --cask claude-code` (но без авто-обновлений).

При первом запуске Claude попросит залогиниться через claude.ai. Обновления ставятся автоматически.

## Pricing

| План | Цена | Модель | Для кого |
|---|---|---|---|
| Pro | $20/мес | Sonnet | Попробовать |
| Max 5x | $100/мес | Opus + Sonnet | Ежедневная работа |
| Max 20x | $200/мес | То же, лимиты x20 | Heavy usage |

Sonnet - быстрая модель для рутинных задач. Opus - для сложного рассуждения. Переключение: `/model`. Для начала хватит Pro за $20.

## Первый запуск

Когда набираешь `claude` в папке проекта, Claude Code:

1. Читает CLAUDE.md если есть (пока нет - ок)
2. Понимает структуру проекта (git, package.json, файлы)
3. Ждёт твой промпт

Набираешь вопрос или задачу - Claude начинает работать. Ты видишь каждый шаг: какие файлы читает, какие команды запускает, что меняет. Всё прозрачно.

Первый промпт может быть простым:

```
Объясни структуру этого проекта. Какие основные модули, как они связаны?
```

## /init - стартовый CLAUDE.md

```
/init
```

Claude проанализирует проект - build system, test framework, паттерны кода - и сгенерирует стартовый CLAUDE.md. Это файл с инструкциями, которые Claude будет читать в начале каждой сессии.

Сгенерированный файл будет базовым. Дальше ты его дорабатываешь руками. Про это - целая глава 3.

## Thinking modes

Claude Code поддерживает разные “глубины” рассуждения:

- **Обычный промпт** - быстрый ответ, рутинные задачи
- **“think”** - расширенное рассуждение
- **“think hard”** - ещё глубже
- **“think harder”** - для сложной архитектуры
- **“ultrathink”** - максимальный бюджет рассуждения

Не нужно ultrathink на каждый промпт. Но когда я планировал миграцию frontmatter на 97 markdown файлов - формат полей, edge cases, batch processing strategy - ultrathink себя оправдал. Для “переименуй переменную” - обычный режим.

Практическое правило: обычный режим для 80% задач, “think” или “think hard” для планирования и архитектуры.

## Plan Mode - исследование без страха

Shift+Tab переключает Plan Mode. В нём Claude:

- Читает файлы
- Отвечает на вопросы
- Анализирует код
- **НЕ вносит изменения**

Идеально для первого погружения в чужой проект, планирования рефакторинга, анализа “что сломается если мы...”.

Я использую Plan Mode каждый раз, когда начинаю работать с незнакомым кодом. На работе - погружение в 4-месячный проект через Plan Mode заняло час вместо нескольких дней. Подробнее об этом - в главе 9.

## Checkpoints - эксперименты без риска

Каждое действие Claude автоматически создаёт checkpoint. Если что-то пошло не так:

- **Esc** - остановить текущее действие
- **Esc + Esc** (двойной) - меню rewind
- **/rewind** - выбрать точку отката

Можно откатить только разговор, только код, или оба. Checkpoints сохраняются между сессиями.

Это снимает страх экспериментов. Раньше я десять раз думал, прежде чем попробовать нестандартный подход - вдруг испортит что-то. С checkpoints: попробовал, не понравилось, откатил за секунду.

## Session management

```
claude --continue    # продолжить последний разговор
claude --resume      # выбрать из недавних сессий
```

Внутри сессии:

- **/clear** - очистить контекст (между несвязанными задачами)
- **/rename** - дать сессии имя ("oauth-migration", "debug-memory-leak")
- **/compact** - сжать контекст, сохранив суть

**Anti-pattern:** мешать разные задачи в одной сессии. Контекст засоряется, Claude начинает путать контексты. Сделал задачу - /clear - следующая задача. Или новый `claude`.

Знакомо? Сессия на час, 15 тем, под конец Claude отвечает невпопад. Это не баг модели - это засорённый контекст.

## Первый полезный результат за 10 минут

**Если есть проект:** `claude` → `/init` → прочитай CLAUDE.md → дополни.

**Если нужно разобраться в чужом коде:** `claude` → Shift+Tab (Plan Mode) → "Объясни архитектуру этого проекта."

**Если есть рутинная задача:** "Добавь ESLint конфиг с правилами для TypeScript" или "Обнови все импорты с relative на absolute paths".

**Если есть markdown-документация:** "Прочитай все файлы в docs/ и сделай summary: что описано, что устарело, что отсутствует." Это уже не "кодинг" - это DX-работа. И Claude делает её отлично.

---

# Глава 3: CLAUDE.md - конституция проекта

> **TL;DR:** 150 строк CLAUDE.md определяют поведение Claude больше, чем выбор модели. Scaffolding важнее модели - но CLAUDE.md advisory, а не enforcement; для гарантий нужны hooks.

---

Первое время с Claude Code я каждую сессию начинал с одних и тех же инструкций: “это Next.js 16 с App Router, Mantine 8, yarn а не npm, статический экспорт, page делегирует в view, context7 MCP подключен для документации”. Каждый раз. 15-20 строк контекста, прежде чем добраться до реальной задачи.

CLAUDE.md решает эту проблему раз и навсегда. 150 строк текста в корне проекта, которые Claude читает автоматически в начале каждого разговора. Написал один раз - работает каждую сессию.

Из всего, что есть в Claude Code, CLAUDE.md - штука с самым высоким leverage для старта. Не единственный компонент (skills, hooks, MCP - следующие главы), но тот, с которого стоит начать.

## Что это такое

CLAUDE.md - постоянный контекст: правила, конвенции, команды, архитектурные решения. Всё, что Claude не может вывести из кода сам.

Без CLAUDE.md каждая сессия начинается с нуля. С ним - Claude уже знает как собрать проект, как запустить тесты, какие конвенции соблюдать, куда не лезть.

## Иерархия

Claude Code читает CLAUDE.md из нескольких мест, от общего к частному:

```
~/.claude/CLAUDE.md          # глобальный (для всех проектов)
    |
./CLAUDE.md                   # проектный (коммитится в git)
    |
./src/CLAUDE.md               # поддиректория (для монорепо)
    |
./.claude/CLAUDE.md           # пользовательский (gitignored)
```

**Глобальный** - твои личные предпочтения для всех проектов.

**Проектный** - коммитится в git, шарится с командой. Build commands, test runners, архитектурные решения.

**Поддиректории** - для монорепо: разные правила для frontend/ и backend/.

**Пользовательский** - личные override для проекта. Gitignored, только для тебя.

## Что включать

```
# Проект

React 19 + TypeScript + Mantine 8. Монорепо с pnpm workspaces.

## Команды

- `pnpm dev` - запустить dev server
- `pnpm test` - Jest, но `pnpm test:e2e` для Playwright
- `pnpm lint` - ESLint + Prettier, запускай перед коммитом

## Архитектура

- API: /src/api/ - tRPC роутеры, zod валидация
- UI: /src/components/ - Mantine компоненты
- State: Zustand stores в /src/stores/, НЕ Redux

## Правила

- НИКОГДА не коммить без подтверждения
- Тесты обязательны для API-роутеров
- Feature branches от main, squash merge

## Gotchas

- pnpm, не npm и не yarn (lockfile другой)
- Node 20+ обязателен (используем ES2023)
- .env.local не в git, пример в .env.example
```

Правило простое: **включай то, что Claude не угадает из кода**. Build commands, нестандартные конвенции, gotchas которые сбивают новичков.

## Что НЕ включать

| Не надо | Почему |
|---|---|
| Стандартные конвенции языка | Claude и так знает TypeScript conventions |
| “Пиши чистый код” | Размытые инструкции = Claude игнорирует |
| Описание каждого файла | Claude сам прочитает при необходимости |
| Подробная API документация | Слишком длинно. Дай ссылку: `@docs/api.md` |
| Часто меняющаяся информация | Устареет, Claude будет следовать устаревшему |
| Code style правила | Задача линтера, не LLM. Настрой hook |

Последний пункт важен. Если хочешь гарантировать formatting - настрой Prettier через hook (глава 5), а не пиши в CLAUDE.md “используй одинарные кавычки”. CLAUDE.md - advisory, Claude может проигнорировать. Hook - детерминированный.

## Лимит ~150 строк

Каждая строка конкурирует за внимание. Длинный CLAUDE.md = ключевые правила тонут в шуме. После ~150 строк Claude начинает пропускать правила в середине файла.

- Ревьюить раз в пару недель
- Если Claude делает правильно без правила - убрать правило
- Длинная документация → @ imports
- Strict requirements → hooks

Это как `.editorconfig` - короткий, конкретный, без объяснений “почему”.

## @ imports

Когда нужен дополнительный контекст без раздувания основного файла:

```
# Проект

Подробнее: @README.md

## Инструкции

- Git workflow: @docs/git-instructions.md
- API conventions: @docs/api-style-guide.md
```

Claude подгрузит файлы по необходимости - не всё сразу.

## /init - старт с нуля

```
/init
```

Claude проанализирует проект и сгенерирует стартовый CLAUDE.md. Не финальная версия - стартовая точка, которую дорабатываешь руками.

## Emphasis для важного

```
IMPORTANT: Никогда не коммить без подтверждения пользователя.
```

Claude обращает больше внимания на выделенные правила. Но не делай каждое правило “IMPORTANT” - эффект размоется.

## CLAUDE.md для DX-инженера

Вот где CLAUDE.md становится инструментом DX, а не просто конфигом:

**Onboarding.** Новый разработчик? CLAUDE.md уже содержит build commands, архитектуру, gotchas. `claude` → сразу в контексте.

**Стандартизация.** Вся команда использует одни правила. Один CLAUDE.md в git.

**Institutional memory.** Практика Boris Cherny: команда добавляет ошибки и best practices через @.claude в PR. Claude наступил на грабли → правило в CLAUDE.md → больше не наступает. CLAUDE.md растёт через реальный опыт команды.

**Context routing.** Для больших проектов - таблица маршрутизации:

```
## Контекст

| Тема | Файл |
|------|------|
| Архитектура | @docs/architecture.md |
| API conventions | @docs/api-style.md |
| Deploy process | @docs/deploy.md |
| Known issues | @docs/gotchas.md |
```

В моём CLAUDE.md для лендинга шесть строк context routing table - и Claude всегда знает куда смотреть для каждой темы. Не загружает всё в контекст, а читает нужный файл когда потребуется.

## advisory vs deterministic

Ключевое ограничение: CLAUDE.md - **advisory**. Claude прочитает, учтёт, но может проигнорировать. Особенно если контекст длинный и правило утонуло.

Если правило **должно** выполняться всегда: - Formatting → hook (PostToolUse) - Блокировка опасных команд → hook (PreToolUse) - Проверка типов → hook (PostToolUse)

CLAUDE.md = “пожалуйста делай так”. Hook = “ты не сможешь сделать иначе”. Разница принципиальная. Об этом - глава 5.

## Карта платформы

CLAUDE.md - первый из четырёх компонентов:

```
CLAUDE.md → Skills → Hooks → MCP
контракт    playbooks  guardrails  руки
advisory    workflow   deterministic  integration
```

**CLAUDE.md** - контракт. Правила, контекст, конвенции. Advisory.

**Skills** - playbooks. Повторяющиеся workflows в одну команду.

**Hooks** - guardrails. Детерминированные проверки и блокировки. Гарантированно.

**MCP** - руки наружу. Подключение к сервисам без CLI/SDK (Notion, Chrome DevTools). Если есть хороший CLI (`gh`, `aws`) - используй его напрямую, это надёжнее.

Порядок важен. CLAUDE.md за 15 минут → первый skill за 10 минут → первый hook за 5 минут → MCP когда нативного CLI недостаточно. Каждый следующий слой - когда предыдущий стал привычным.

---

# Глава 4: Skills - переиспользуемые workflows

> **TL;DR:** Skill - один раз описанный workflow вместо повторного написания промптов. Но если задача решается bash-скриптом - реши скриптом. Skill нужен когда в цепочке есть шаги с AI-рассуждением.

---

Каждый раз, когда я добавлял новую страницу на лендинг, я писал промпт на 15 строк: “создай view-компонент в src/views/, добавь page.tsx в app/, обнови Layout с навигацией, напиши CSS Module, проверь что Storybook story работает, запусти typecheck”. Через неделю забывал про Storybook. Через две - про обновление навигации в Layout. Через месяц писал промпт заново, потому что не помнил порядок шагов.

Skill - это один раз описанный workflow, который вызывается через `/skill-name`. Всегда одинаково. С правильными шагами. В правильном порядке.

## Формат SKILL.md

Skill - файл `SKILL.md` в директории `.claude/skills/`:

```
.claude/skills/
  deploy/SKILL.md
  fix-issue/SKILL.md
  create-component/SKILL.md
```

Простейший skill:

```
---
name: deploy
description: Deploy current branch to staging
---
# Deploy to staging

1. Run `pnpm build`
2. Run `pnpm test`
3. If tests pass, run `./scripts/deploy-staging.sh`
4. Report the deployment URL
```

Вызов: `/deploy`. YAML frontmatter с именем и описанием + markdown с инструкциями. Все доступные skills видны через `/` - Claude Code покажет список с описаниями.

## Ключевые параметры

**Auto vs manual trigger.** По умолчанию Claude может применить skill автоматически. Для production-affecting workflows:

```
---
name: deploy
description: Deploy current branch to staging
disable-model-invocation: true
---
```

`disable-model-invocation: true` - только ручной вызов.

**$ARGUMENTS - параметризация:**

```
---
name: fix-issue
description: Analyze and fix a GitHub issue
disable-model-invocation: true
---
Fix the GitHub issue: $ARGUMENTS

1. Use `gh issue view $ARGUMENTS` to get details
2. Understand the root cause
3. Implement the fix
4. Write tests
5. Create a commit
```

Вызов: `/fix-issue 1234`. Один skill покрывает любой issue.

**allowed-tools - ограничение:**

```
---
name: review-code
description: Review code for quality issues
allowed-tools: Read, Grep, Glob
---
```

Claude сможет только читать - не редактировать, не запускать команды.

**Hooks внутри skills** - scoped automation:

```
---
name: careful-refactor
hooks:
  PostToolUse:
    - matcher: "Edit"
      hooks:
        - type: command
          command: "npx tsc --noEmit"
---
```

TypeScript проверка после каждого Edit - но только внутри этого skill.

## Где хранить

```
~/.claude/skills/       # глобальные (для всех проектов)
.claude/skills/         # проектные (коммитятся, для команды)
```

## Когда skill, а когда скрипт

Принцип, который мы развиваем в главе 5: код > AI. Если задача полностью формализуема - пиши скрипт, не skill. Skill нужен когда в workflow есть шаги, требующие AI-рассуждения (анализ, генерация текста, принятие решений).

## Пример: skills для ежедневной разработки

Несколько skills из моего рабочего набора:

**Создание:** - `/create-page` - новая страница: view-компонент, page.tsx в App Router, навигация, CSS Module - `/create-component` - компонент по шаблону: файл, тесты, Storybook story, экспорт из индекса - `/fix-issue` - по номеру GitHub issue: читает описание, находит код, фиксит, пишет тесты

**Качество:** - `/review` - code review: читает diff, ищет проблемы с типами, Mantine patterns, edge cases - `/check-deps` - проверка зависимостей: устаревшие пакеты, breaking changes, security - `/sync-docs` - синхронизирует docs/ с текущим кодом (API, архитектура, README)

**Рутина:** - `/deploy-staging` - сборка, тесты, деплой, URL - `/create-pr` - diff текущей ветки, описание, чеклист ревью, линк на issue

Каждый решает конкретную повторяющуюся задачу. До skills я каждый раз писал промпт заново. Теперь `/fix-issue 1234` - три секунды.

## Пример: от бага до деплоя

Цепочка skills покрывает полный цикл разработки:

```
/fix-issue 1234
    → читает описание issue через gh
    → находит связанный код через grep
    → реализует фикс, добавляет тесты
    → запускает typecheck и lint
    → создаёт коммит с ссылкой на issue

/create-pr
    → собирает diff текущей ветки
    → генерирует описание PR из коммитов
    → добавляет чеклист ревью
    → линкует issue

/deploy-staging
    → запускает yarn build и yarn test
    → деплоит на staging
    → возвращает URL для проверки
```

Три команды. Каждая заменяет 15-30 минут ручной работы. И каждый раз одинаковый формат, потому что workflow зашит в skill.

## Skills для команды

Skills коммитятся в `.claude/skills/` - значит доступны всем.

Сценарий: новый разработчик приходит в команду. В репозитории:

```
.claude/skills/
  fix-issue/SKILL.md       # /fix-issue 1234
  create-pr/SKILL.md       # /create-pr
  deploy-staging/SKILL.md  # /deploy-staging
  onboard/SKILL.md         # /onboard
```

Новичок набирает `/onboard` - Claude проводит через архитектуру, показывает ключевые файлы, объясняет конвенции. Не “прочитай wiki на 50 страниц”, а интерактивный walkthrough.

## Чеклист

- ☐ Определи 2-3 повторяющихся workflow
- ☐ Создай SKILL.md (начни без hooks и allowed-tools)
- ☐ Production-affecting skills: `disable-model-invocation: true`
- ☐ Team workflows: положи в `.claude/skills/` (коммитится)

---

# Глава 5: Hooks - детерминированная автоматизация

> **TL;DR:** Hooks дают детерминизм внутри AI-системы. Всё что можно гарантировать кодом - гарантируй кодом, а не промптом. Exit code 2 блокирует действие, exit 0 добавляет feedback. Начни с auto-format - один hook, максимальный эффект.

---

В главе 3 я написал: CLAUDE.md - advisory. Claude прочитает, учтёт, но может проигнорировать. Три раза я писал в CLAUDE.md “используй Mantine компоненты, не голый HTML”. Три раза Claude генерировал `<div>` вместо `Box` и `<a>` вместо `Anchor`. Сделал hook, который после каждого Edit проверяет файл на голые HTML-теги - проблема исчезла навсегда.

Hooks решают эту проблему системно. Hook - это код, который выполняется гарантированно в определённой точке lifecycle Claude Code. Не “пожалуйста, отформатируй после редактирования”, а shell-скрипт, который запускает Prettier после каждого Edit. Каждый раз. Без исключений.

Для DX-инженера это ключевое: ты строишь системы, которые должны работать надёжно. “Обычно работает” - не подходит.

## Принцип: код > AI

Daniel Miessler и я пришли к одному выводу независимо друг от друга:

> Всё, что можно сделать детерминированно - делай детерминированно. AI подключай только когда нужно рассуждение.

Hooks - воплощение этого принципа прямо внутри Claude Code. Ты не просишь AI “не забудь отформатировать” - ты запускаешь Prettier автоматически. Не просишь “не трогай .env” - ты блокируешь доступ через exit code.

Есть ещё одна сторона: **автоматизируемое = делаемое**. То, что требует ручного усилия - рано или поздно забудется. То, что работает автоматически - работает всегда. Hooks превращают “надо бы” в “уже сделано”.

### Пример: codemod вместо промпта

Нужно мигрировать 200 файлов с одного API на другой. Два варианта:

**AI-подход:** “Пройди по всем файлам и замени `oldApi.fetch()` на `newApi.request()`”. Claude будет открывать файлы один за другим, тратить токены, иногда пропускать edge cases.

**Детерминированный подход:** jscodeshift - AST-трансформация. Парсит код в дерево, находит нужные вызовы, заменяет. Каждый файл. Каждый раз. Без токенов.

```
npx jscodeshift -t transforms/migrate-api.ts src/**/*.ts
```

Ключевое - **тестируемость**. Codemod можно покрыть тестами: входной файл → трансформация → ожидаемый результат. Проверено на 5 примерах - запущено на тысяче.

Claude Code полезен на другом уровне: **написать сам codemod**. AI создаёт инструмент, инструмент работает детерминированно. Тот же принцип для ESLint rules, Prettier plugins, PostCSS transforms.

### Пример: PDF-парсинг без LLM

Мне нужно было извлекать данные из банковских PDF-выписок. Первый подход - локальные LLM: модель читает страницу, извлекает суммы и даты. Работало, но медленно (минута+ на файл), и результаты были нестабильные - приходилось запускать несколько моделей параллельно и сравнивать.

Начал добавлять Python-проверки для валидации LLM-ответов. И в какой-то момент понял: проверки покрывают всю логику. Python с регулярками и табличным парсингом делает то же самое за секунды, стабильно, без GPU.

Итог: полностью детерминированный pipeline. LLM не нужна. Быстрее, надёжнее, воспроизводимо. Классический путь: начал с AI, закончил кодом.

## Как устроены hooks

### Lifecycle events

| Event | Когда срабатывает | Можно блокировать? |
|---|---|---|
| `SessionStart` | Начало или возобновление сессии | Нет |
| `UserPromptSubmit` | Ты нажал Enter на промпте | Да |
| `PreToolUse` | Claude собирается вызвать инструмент | Да |
| `PostToolUse` | Инструмент успешно отработал | Нет |
| `Stop` | Claude закончил ответ | Да |
| `Notification` | Claude хочет отправить уведомление | Нет |
| `PreCompact` | Перед сжатием контекста | Нет |

Полный список событий - в документации. Здесь самые полезные для DX.

**“Можно блокировать”** - hook возвращает exit code 2 и действие отменяется. PreToolUse с exit 2 = Claude не сможет выполнить этот инструмент.

### 4 типа handlers

**Command** - shell-скрипт. Самый частый:

```
{ "type": "command", "command": "node scripts/format-check.js" }
```

**HTTP** - POST-запрос. Для интеграции с внешними сервисами.

**Prompt** - single-turn LLM evaluation. Быстрая проверка через Haiku:

```
{ "type": "prompt", "prompt": "Is this change complete? Reply YES or NO." }
```

**Agent** - multi-turn subagent. Для сложных проверок (code review, security audit).

Для DX-автоматизаций чаще всего хватает command.

### Exit codes

- **Exit 0** - OK, продолжай. stdout добавляется в контекст Claude
- **Exit 2** - СТОП. Действие блокируется, stderr отдаётся Claude как feedback
- **Любой другой** - warning, действие продолжается

### Matchers

Regex-паттерн, фильтрующий когда hook срабатывает:

```
"Bash"           # только Bash-команды
"Edit|Write"     # Edit или Write
"mcp__github__*" # все инструменты GitHub MCP
```

Без матчера - срабатывает на всё.

### Где настраивать

```
~/.claude/settings.json           # глобально (все проекты)
.claude/settings.json             # проект (коммитится)
.claude/settings.local.json       # проект (gitignored)
```

Или интерактивно: `/hooks` в Claude Code.

## 3 core рецепта

Три hooks, которые покрывают 80% потребностей. Остальные (context priming, TypeScript check, backup, Telegram notification) - в Appendix.

### 1. Auto-format после редактирования

Claude не всегда форматирует код по стандартам проекта. CLAUDE.md тут не поможет - это задача для детерминированного инструмента.

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "node scripts/hooks/auto-format.js"
      }]
    }]
  }
}
```

`scripts/hooks/auto-format.js`:

```
const { execSync } = require('child_process');

const input = JSON.parse(require('fs').readFileSync('/dev/stdin', 'utf8'));
const filePath = input.tool_input?.file_path;

if (!filePath) process.exit(0);

const ext = filePath.split('.').pop();
if (['js', 'ts', 'tsx', 'jsx', 'json', 'css'].includes(ext)) {
  try {
    execSync(`npx prettier --write "${filePath}"`, { stdio: 'pipe' });
  } catch (e) {
    // Prettier не установлен - не блокируем
  }
}
```

Boris Cherny, Head of Claude Code в Anthropic, использует этот hook в каждом проекте.

### 2. Permissions - встроенная система разрешений

Для блокировки и авторазрешения не обязательно писать hooks. В `settings.json` есть встроенная система permissions:

```
{
  "permissions": {
    "allow": [
      "WebFetch",
      "WebSearch",
      "Bash(git status:*)",
      "Bash(git log:*)",
      "Bash(git diff:*)",
      "Bash(ls:*)",
      "Bash(cat:*)",
      "Bash(which:*)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)"
    ]
  }
}
```

`allow` - Claude выполняет без переспроса. `deny` - Claude не сможет выполнить вообще. Всё остальное - спрашивает подтверждение.

На практике это первое, что стоит настроить. Без allow Claude переспрашивает на каждый `ls`, `git status`, чтение документации - по 300 раз за сессию. С allow для безопасных команд работа становится плавной.

Для более сложной логики (паттерны, условия, кастомные проверки) - PreToolUse hook:

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "bash .claude/hooks/block-dangerous.sh"
      }]
    }]
  }
}
```

Exit code 2 = Claude не сможет выполнить команду. Даже если очень хочет.

### 3. Desktop notification при ожидании

Запустил длинную задачу, переключился - забыл, что Claude ждёт input. Одна строчка решает:

```
{
  "hooks": {
    "Notification": [{
      "hooks": [{
        "type": "command",
        "command": "osascript -e 'display notification \"Claude Code ждёт ответа\"
        with title \"Claude Code\"'"
      }]
    }]
  }
}
```

Простейший hook - но экономит кучу времени при работе с несколькими сессиями.

## Комбинирование hooks

Мой рабочий набор - 5 hooks:

1. **SessionStart** - загрузка контекста (приоритеты + активные задачи)
2. **PostToolUse** - auto-format после Edit/Write
3. **PreToolUse** - блокировка опасных Bash-команд
4. **Notification** - macOS notification при ожидании input
5. **Stop** - сохранение session label для идентификации вкладок

Начинай с одного. Auto-format - лучший первый hook: максимальный эффект, минимальный риск.

## Hooks для команды

Hooks коммитятся в `.claude/settings.json` - значит работают для всех в команде.

Это не “рекомендация” в wiki, которую никто не читает. Это enforcement. Каждый разработчик, использующий Claude Code в этом репозитории, автоматически получает: - Auto-format (единый стиль) - Блокировку опасных команд (safety) - Notification (awareness)

Для DX-инженера это dream: правила, которые невозможно обойти по забывчивости.

## За пределами basics

В community уже десятки documented hooks. Самые интересные:

- **Dippy** - AST-based analysis для безопасного auto-approve команд
- **afk-code** - управление Claude Code через Telegram
- **claude-mem** - persistent memory через SQLite + vector DB

Полный каталог и дополнительные рецепты (context priming, TypeScript check, backup transcript, Telegram push) - в Appendix.

---

# Глава 6: MCP - подключение к внешнему миру

> **TL;DR:** MCP - стандартный разъём для подключения Claude к внешним сервисам. Но если у сервиса есть хороший SDK или CLI - используй их. Это надёжнее, функциональнее и соответствует принципу детерминированности. MCP - для сервисов без CLI и для сложных интеграций с OAuth.

---

CLAUDE.md говорит Claude как работать. Hooks автоматизируют события. Skills кодифицируют workflows. Но всё это - внутри проекта. А реальная DX-работа часто требует доступа наружу: прочитать issue в Jira, посмотреть PR на GitHub, проверить метрики, обновить запись в Notion.

MCP (Model Context Protocol) решает эту проблему. Думай о нём как об USB: не нужно понимать протокол, чтобы подключить мышку. Подключил - работает. MCP-сервер для GitHub - и Claude видит PR и issues. Для PostgreSQL - и Claude читает таблицы. Один стандарт, сотни сервисов.

Это не проприетарная штука Anthropic. OpenAI принял MCP в 2025, другие вендоры подтянулись. Экосистема растёт экспоненциально - тысячи серверов за первый год. Индустриальный стандарт, который будет работать независимо от того, какой AI-инструмент ты используешь завтра.

## Как работает

MCP-сервер - маленькая программа, которая предоставляет Claude набор инструментов. GitHub MCP сервер даёт `list_prs`, `create_issue`, `search_code`. PostgreSQL - `query`, `list_tables`.

Два транспорта:

**HTTP** - удалённый сервер. Для облачных сервисов:

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

**stdio** - локальный процесс. Для доступа к локальным ресурсам:

```
claude mcp add github -- npx -y @modelcontextprotocol/server-github
```

Подключил - и Claude видит новые инструменты. Без перезапуска, без конфигурации кода.

## Must-have серверы

Два сервера, которые стоит подключить в первый день:

**GitHub** - PR, issues, code search. Если работаешь с GitHub - must have. Skill `/fix-issue` читает issue напрямую, создаёт PR с фиксом, всё в одной сессии.

**Context7** - актуальная документация любой библиотеки.

```
claude mcp add context7 -- npx -y @upstash/context7-mcp
```

Зачем это нужно: Claude знает то, что было в training data. Для стабильных библиотек - достаточно. Но для быстро меняющихся фреймворков - нет. Мой кейс: работаю с Mantine 8, которая вышла недавно. Claude уверенно предлагает API из Mantine 7 - компоненты переименованы, пропсы изменились, а он этого не знает. Context7 даёт live-доступ к текущей документации, и Claude использует правильные API.

Это не экзотическая проблема. Любой фреймворк с активным развитием - React 19, Next.js 15, Svelte 5 - имеет breaking changes, которых нет в training data.

## Другие полезные серверы

| Сервер | Что даёт | DX-применение |
|---|---|---|
| Chrome DevTools | Скриншоты, DOM, console, network | Visual testing, debugging UI |
| PostgreSQL / Supabase | SQL-запросы, схема | Анализ данных без переключения |
| Notion / Linear / Jira | Задачи, документы | Чтение эпиков и подтикетов |
| Sentry | Ошибки, стектрейсы | Debugging production issues |
| Cloudflare | DNS, Workers, Pages | Deploy и настройка |

## Scopes

```
claude mcp add --scope local github ...    # только я, этот проект
claude mcp add --scope project github ...  # команда (в .mcp.json, коммитится)
claude mcp add --scope user github ...     # я, все проекты
```

`--scope project` кладёт конфиг в `.mcp.json` - коммитится в git. Вся команда получает одни и те же интеграции. Для DX-инженера это способ стандартизировать tooling: подключил MCP-серверы в `.mcp.json`, закоммитил - каждый новый разработчик сразу получает доступ ко всем нужным сервисам.

Альтернатива CLI - прямое редактирование `.mcp.json`. Environment variables раскрываются автоматически - секреты не попадают в git.

## SDK/CLI > MCP

Принцип из прошлых глав: детерминизм > AI. Это относится и к MCP - даже сильнее, чем кажется.

Claude Code работает в терминале. Ему доступны все CLI-инструменты и SDK на машине. `gh` для GitHub, `aws` для AWS, `jq` для JSON, `az` для Azure DevOps. И нативный CLI/SDK почти всегда лучше MCP-сервера по трём причинам:

**1. Надёжность.** CLI - детерминированный инструмент: один вход, один выход, предсказуемое поведение. MCP-сервер - посредник между Claude и API, часто community-maintained, с собственными багами и ограничениями.

**2. Полнота API.** MCP-серверы обычно покрывают 20-30% API сервиса - самые частые операции. Нативный CLI/SDK даёт доступ ко всему. `gh` может всё, что может GitHub API. MCP-сервер для GitHub - подмножество.

**3. Детерминированность.** CLI-команда - это код. Её можно зашить в hook, skill, скрипт. MCP-вызов - запрос через AI, который может интерпретировать его по-разному. Это противоречит принципу 80/20.

Пример из главы 9: для погружения в проект через issue tracker не нужен MCP. Claude использует `az boards` CLI напрямую - запрашивает work items, PR, диффы. Работает, не требует настройки MCP-сервера, результат тот же, а зачастую даже лучше.

**Когда использовать MCP:** - Сервисов без CLI/SDK (Chrome DevTools, браузерная автоматизация) - Сложных интеграций с OAuth, где CLI отсутствует (Notion, Figma) - Задач, где нужен rich structured output с навигацией

**Когда использовать CLI/SDK:** - Всегда, когда есть хороший CLI ( `gh` , `aws` , `az` , `jira` , `kubectl` ) - Всё, что можно скриптовать в hook или skill - Простых одноразовых запросов

Правило: прежде чем подключать MCP-сервер, проверь - есть ли у сервиса CLI. Если да, и он хорошо поддерживается - используй CLI. MCP добавляй только для того, чего CLI не покрывает.

## MCP + Skills = сильная комбинация

Когда MCP оправдан - он хорошо комбинируется со skills:

```
---
name: investigate-issue
description: Deep investigation of a production issue
---
1. Read the Sentry error via MCP: get stack trace and affected users
2. Find the relevant code in the codebase
3. Check recent PRs that touched those files (GitHub MCP)
4. Check deployment history (Cloudflare MCP)
5. Produce a root cause analysis with timeline
```

Skill определяет workflow, MCP даёт доступ к данным. Одна команда - полное расследование production issue.

## Tool Search

Когда подключено много MCP-серверов, описания инструментов съедают контекст. Tool Search включается автоматически при >10% окна контекста. Claude ищет нужный инструмент по описанию вместо загрузки всех. На практике это значит: можно подключить 10-15 серверов без деградации качества.

---

# Глава 7: Продвинутая кастомизация

> **TL;DR:** Subagents, worktrees, plugins - каждый уровень подключай только когда предыдущий стал тесен. Эта глава - меню, не чеклист.

---

Главы 3-6 покрывают 90% повседневных нужд: CLAUDE.md, hooks, skills, MCP. Эта глава - про оставшиеся 10%.

Я дошёл до subagents через месяц после начала работы с Claude Code. До worktrees - через полтора. До сих пор не использую Agent Teams. Нет проблемы - не нужен инструмент.

## Subagents: изолированные контексты

**Проблема:** исследуешь большую кодовую базу, результаты поиска забивают контекст основной задачи.

**Решение:** Claude запускает subagent - отдельный instance с собственным контекстом. Subagent исследует, возвращает summary. Основной контекст чист.

Два типа: - **Agent tool** - синхронный, ждёшь результат - **Task tool** - асинхронный, продолжаешь работать

Кастомные subagents в `.claude/agents/`:

```
---
name: security-reviewer
description: Reviews code for security vulnerabilities
tools: Read, Grep, Glob
model: opus
---
Review the code for:
- Injection vulnerabilities
- Authentication flaws
- Secrets in code
Provide specific line references and fixes.
```

На практике я чаще всего использую subagents для research - когда нужно прочитать 20 файлов и вернуть summary, не засоряя основной контекст.

## Worktrees: git-изоляция

**Проблема:** два агента работают параллельно и конфликтуют при изменении файлов.

```
claude --worktree
```

Создаёт изолированный git checkout. Каждый агент работает в своей копии. Auto-cleanup если нет изменений, merge обратно если есть.

Когда это полезно: запустил один агент на рефакторинг модуля A, другой - на модуль B. Без worktrees они могут редактировать общие файлы (imports, configs) и ломать друг другу работу. С worktrees - каждый в своей ветке, конфликты видны при merge, а не в процессе.

Ограничение: если оба агента трогают одни и те же файлы - merge conflict неизбежен. Worktrees изолируют, не магически решают конфликты. Для задач на пересекающихся файлах лучше sequential подход.

## Auto-memory

**Проблема:** каждую сессию Claude забывает что обсуждали в прошлый раз.

Auto-memory: Claude сохраняет паттерны, предпочтения, архитектурные заметки в `MEMORY.md`.

- Хранение: `~/.claude/projects/<project>/memory/`
- Лимит: 200 строк основного файла + topic files
- Управление: `/memory`

У меня MEMORY.md вырос до 200 строк за три недели - Claude записывал решения, конвенции, результаты исследований. Полезно для long-running projects, где контекст накапливается между сессиями.

Встроенной памяти пока хватает, но есть и внешние решения - например, mem0. Они дают больше контроля: приоритизация воспоминаний, semantic search по истории, интеграция с несколькими агентами. Имеет смысл смотреть в эту сторону, когда встроенный MEMORY.md станет тесен или когда нужна память, общая для нескольких инструментов.

## Non-interactive mode

**Проблема:** нужно запустить Claude из скрипта, CI/CD, cron job.

```
claude -p "Explain what this project does"
claude -p "List all API endpoints" --output-format json
```

**Fan-out pattern** - batch обработка:

```
for file in $(cat files-to-migrate.txt); do
  claude -p "Migrate $file from CommonJS to ESM. Return OK or FAIL." \
    --allowedTools "Edit,Bash(npm test *)"
done
```

`--allowedTools` ограничивает что Claude может делать в batch mode.

Пример: у меня есть skill `/progress-report`, который в non-interactive mode читает git log по всем репозиториям, группирует по неделям и генерирует changelog за месяц. Одна команда вместо ручного обхода проектов.

## Sandboxing и Permissions

OS-level изоляция. На macOS включена по умолчанию (Seatbelt):

- Write access: только текущая директория
- Read access: весь компьютер кроме заблокированных путей

`/sandbox` для настройки. Дополнительно - `permissions` в settings.json (глава 5): `allow` для безопасных инструментов без переспроса, `deny` для полной блокировки. В моём опыте это убирает подавляющее большинство permission prompts.

## Plugins

Plugin объединяет skills + hooks + MCP + agents в один устанавливаемый пакет. Устанавливаются из npm registry с version pinning, browse через `/plugin`.

Пока экосистема plugins молодая - большинство задач решаются skills + hooks. Plugins станут актуальнее когда появятся стандартные пакеты для типовых workflows (linting presets, deploy pipelines, team onboarding kits).

## Agent Teams и Remote Control

Обе фичи в research preview (на момент написания).

**Agent Teams** - один lead координирует несколько teammates, каждый с собственным контекстом. В теории хорош для параллельного research с нескольких сторон. На практике overhead координации пока высок, и для большинства задач subagents проще и предсказуемее.

**Remote Control** - мост между локальной сессией и claude.ai/code или мобильным приложением. Идея: запустил долгую задачу в терминале → закрыл ноутбук → управляешь с телефона.

## Лестница расширения

```
CLAUDE.md          # начни здесь
    |
Hooks              # когда нужен детерминизм
    |
Skills             # когда повторяешь workflows
    |
MCP                # когда нужны внешние данные
    |
Subagents          # когда контекст не вмещает
    |
Agent Teams        # когда задача > одной сессии
    |
Plugins            # когда шаришь с другими
```

---

**Конец Части 1.** Ты знаешь как настроить Claude Code от базовой установки до продвинутой платформы.

**Часть 2** - про другое: как использовать всё это для улучшения Developer Experience. Начнём с неудобной правды про AI и продуктивность.

# Глава 8: Парадокс AI-продуктивности

> **TL;DR:** AI ускоряет индивидуалов, но замедляет организации без правильных процессов. Сначала процессы - потом инструменты. Bottom-Up DevEx: начни с одного инженера и одного CLAUDE.md, измерь, расширь.

---

До Claude Code мой GitHub heatmap был почти пустым. Не потому что было мало идей - идей было много. Но между “придумал” и “закоммитил” лежала пропасть. Идея → анализ → ещё анализ → “надо бы ещё подумать” → идея устарела.

За первый месяц с Claude Code - ~200 коммитов. Зелёный heatmap вместо пустоты. Не потому что стал работать по 16 часов.

Но прежде чем объяснять как - нужно честно разобраться, когда AI помогает, а когда нет. Потому что данные говорят неожиданное.

## METR: AI замедляет опытных разработчиков

В 2025 году METR провёл рандомизированное контролируемое исследование (metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study). 16 опытных разработчиков, 246 реальных задач в зрелых open-source проектах (22K+ stars, 1M+ строк кода).

**Результат: AI замедлил работу на 19%.**

Не ускорил. Замедлил.

Самое поразительное: до начала эксперимента разработчики ожидали ускорение на 24%. После завершения - оценивали ускорение на 20%. А реальное измерение показало замедление на 19%.

Почему: опытные разработчики на знакомых кодовых базах тратят больше времени на review, тестирование и отклонение AI-генераций, чем сэкономили бы сделав сами.

Важная оговорка: в феврале 2026 METR обновил дизайн эксперимента и обнаружил, что участники отказываются работать без AI - сигнал становится ненадёжным. AI-

инструменты настолько вошли в привычку, что чистый контрольный эксперимент уже сложно провести. Но сам вывод про overhead для опытных devs на знакомом коде остаётся значимым.

## Индивидуально лучше, организационно - никак

Телеметрия Faros AI (faros.ai/blog/ai-software-engineering) по 10,000+ разработчикам из 1,255 команд показала ещё один парадокс. На уровне отдельных разработчиков:

- Tasks completed: **+21%**
- Pull requests merged: **+98%**
- Daily PRs: **+47%**

Выглядит отлично. Но на уровне организации:

- Code review time: **+91%**
- Pull request size: **+154%**
- Bug rates: **+9%**
- Delivery metrics: **flat**

Разработчики генерируют больше кода - но PR стали в 2.5 раза больше, review занимает почти вдвое дольше, багов стало больше, а итоговая скорость доставки не изменилась.

DORA Report 2025 (dora.dev/research/2025/dora-report) независимо пришёл к тому же выводу:

> “AI’s primary role is that of an amplifier, magnifying the strengths of high-performing organizations and the dysfunctions of struggling ones.”

AI усиливает то, что уже есть. Хорошие процессы станут лучше. Плохие станут хуже. Если в команде и без AI были проблемы с code review - AI сделает PR в 2.5 раза больше, и review утонет.

## Когда AI реально помогает

Данные не однозначно негативные. AI помогает в конкретных сценариях:

**Незнакомый код.** Onboarding, чужой проект, legacy codebase. Там, где ты тратил часы на чтение - Claude проведёт за минуты. По данным enterprise-кейсов Anthropic, juniors на integration tasks показывают значительное ускорение.

**Исследование и документация.** Синтез из множества источников, структурирование знаний, поиск по большим объёмам текста. Большинство разработчиков уже используют AI для документации.

**Рутинные и однотипные задачи.** Миграции, рефакторинг по паттерну, генерация boilerplate. То, что скучно и не требует глубокого рассуждения.

**Scaffolding и автоматизация.** CLAUDE.md, hooks, skills, скрипты. AI помогает СОЗДАТЬ автоматизацию, которая потом работает детерминированно.

## Когда AI мешает

**Опытный dev в знакомом коде.** METR это доказал. Overhead review > benefit.

**Плохие процессы.** AI генерирует больше кода, но если нет хорошего code review, CI/CD, тестов - просто больше кода с большим количеством проблем.

**Нет verification.** “Claude сделал - наверное правильно.” Trust-then-verify gap. Boris Cherny: “Give Claude a way to verify its work. 2-3x quality с feedback loops.” Без verification - quality падает.

## 7 AI-amplifying capabilities

DORA выделил 7 вещей, которые должны быть на месте ДО того, как AI начнёт помогать:

1. **Clear AI policies** - команда знает что можно, что нельзя
2. **Healthy data ecosystems** - данные структурированы и доступны
3. **AI-accessible internal data** - AI может читать документацию, issues, метрики
4. **Strong version control** - git, branching, review процесс
5. **Small batch work** - маленькие задачи, маленькие PR
6. **User-centric product strategy** - понимание зачем
7. **Quality internal platforms** - CI/CD, dev environments, tooling

Узнаёшь? Это DX. Все семь пунктов - характеристики хорошего developer experience. AI усиливает то, что есть. Если DX хороший - AI делает его отличным. Если DX плохой - AI делает его ещё хуже.

## Bottom-Up DevEx: другой подход к внедрению

Большинство компаний внедряют AI top-down: “купили 500 лицензий, разберётесь”. Данные показывают, что это не работает.

Подход этой книги - Bottom-Up DevEx. Термин вырос из моей статьи Why Developers Don’t Know What Dev Ex Is (The New Stack) и развит в работе “The Bottom-Up Approach to Developer Experience” (2026). В ней - 4-этапный процесс:

1. **Friction Log** - инженер документирует friction points в повседневной работе
2. **DX-Champion Analysis** - анализ: что из этого можно исправить снизу, без бюджета
3. **Improvement Initiatives** - конкретные улучшения, измеримые результаты
4. **Leadership Engagement** - доказанные результаты масштабируются на команду

Теоретическая база: DevEx (Fagerholm & Münch), Cognitive Load Theory, Socio-Technical Congruence. Ключевой инсайт: bottom-up улучшения работают как “socio-technical probes” - локальные фиксы вскрывают системные проблемы организации.

В этой книге фреймворк применён к Claude Code. Пять практических принципов:

1. **Начни от IC** - один инженер, один CLAUDE.md, один hook
2. **Измерь** - baseline survey до внедрения, не после
3. **Расширь** - от одного человека к команде, не наоборот
4. **Детерминизм first** - 80% hooks и скрипты, 20% AI
5. **Честно про границы** - AI усиливает, не чинит

Friction Log из статьи → CLAUDE.md gotchas + hooks в книге. DX-Champion → adoption champion (глава 12). Improvement Initiatives → skills и workflows. Leadership Engagement → measurement (глава 13).

Не “поставил Claude Code и всё стало лучше”. А “зафиксировал friction, выстроил процессы, встроил Claude Code как усилитель, измерил результат”.

Вот почему мы начали с Части 1 (scaffolding) прежде чем перейти к Части 2 (применение). Порядок важен.

## Личный опыт: от paralysis к shipping

| | Было | Стало (первый месяц с Claude Code) |
|---|---|---|
| **Коммитов/месяц** | ~0 (личные проекты не трогал годами) | ~200 |
| **Shipped products** | 0 (всё “в процессе”) | Лендинг, автоматизации, knowledge base |
| **Что изменилось** | Analysis paralysis, “ещё подумаю” | Plan Mode на старте, hooks на финише |
| **Роль AI** | - | AI пишет код, я управляю направлением |

За этой таблицей - история глубже чем “установил инструмент”.

До Claude Code я полтора года использовал ChatGPT. 1,382 диалога. Проанализировал экспорт - и увидел закономерность: описывал одни и те же паттерны с возрастающей глубиной понимания - и нулевым изменением в поведении. ChatGPT стал зеркалом для рефлексии, но не инструментом для действия.

Claude Code - другое. Не зеркало, а execution partner. Разница - в артефактах: git-коммиты, shipped продукты, опубликованные посты. Friction снят в двух конкретных местах:

**На старте:** “о чём думаю уже месяц” → Plan Mode, структурирование → “вот план, первый шаг понятен” → начал.

**На финише:** “код написан, но ещё надо отформатировать, проверить типы, написать commit message, лень...” → hooks делают это автоматически → закоммитил.

Между стартом и финишем - я практически не пишу код руками. Claude Code пишет, я направляю: архитектура, решения, приоритеты, review. Мои знания и опыт определяют что и как строить. AI - руки, я - голова. Это не “AI делает всё за меня”. Это “10 лет опыта наконец конвертируются в shipped продукты без friction на last mile”.

Можно ли отделить Claude Code от мотивационного всплеска? Честно - нет. Correlation is not causation. Но friction на старте и финише - измеримый. Время от “идея” до “первый коммит” сократилось с дней до минут. Это не ощущение - это git log.

---

# Глава 9: Погружение в проект за час

> **TL;DR:** Незнакомый код - sweet spot для AI. Claude Code + CLI issue tracker'a + Plan Mode дают базовое понимание 4-месячного проекта за час. Ключ - just-in-time learning: не "прочитай всё", а "дай то, что нужно сейчас".

---

Onboarding в проект - одна из самых дорогих операций в разработке. Новый разработчик, новая команда, или ты вернулся к проекту через полгода - каждый раз одно и то же: десятки тикетов, сотни PR, документация которая частично устарела, чаты которые никто не читает целиком.

Глава 8 показала: AI хуже всего работает для опытного dev в знакомом коде. А вот для погружения в незнакомый код - ровно наоборот. Это sweet spot. По данным enterprise-кейсов Anthropic, juniors на integration tasks в незнакомых проектах показывают значительное ускорение. Не потому что AI пишет за них - а потому что отвечает на вопросы, которые раньше требовали часа чтения кода или отвлечения коллеги.

## Кейс: 4 месяца за час

Типичная ситуация: инженер присоединяется к проекту, который идёт уже 4 месяца. Больше 40 тикетов в issue tracker'e, PR в каждом, связи между задачами, несколько разработчиков параллельно.

Классический подход: открыть epic, читать тикеты один за одним, переходить в PR, смотреть диффы, пытаться выстроить картину в голове. 3-5 дней, если повезёт.

Что можно сделать вместо этого:

1. Claude Code через CLI issue tracker'a получает доступ к тикетам, PR, диффам напрямую из терминала
2. Читает epic и все подтикеты - каждый work item, каждое описание
3. Читает все PR в этих задачах - каждый diff, каждый review comment
4. Выстраивает pipeline: что сделано, что в процессе, кто чем занимается, какие зависимости между задачами

5. Генерирует HTML-отчёт с навигацией по всему epic

| | Было | Стало |
|---|---|---|
| **Время погружения** | 3-5 дней чтения тикетов, PR, документации | ~1 час структурированного анализа |
| **Формат** | Ручной обход десятков тикетов | HTML-отчёт с pipeline, статусами, навигацией |
| **Покрытие** | Что успеешь прочитать | Каждый тикет, каждый PR, каждый diff |
| **Первый полезный вопрос** | День 2-3 | Через час |

Результат: базовое понимание проекта за час. Не глубокое - базовое. Но достаточное, чтобы на следующий день задавать предметные вопросы вместо “а что тут вообще происходит?”

Побочный эффект: при системном обходе всех тикетов Claude может находить несоответствия, которые человек заметил бы не сразу. Тикеты в статусе “done” с незамерженным PR. Задачи без assignee. Зависимости, которые не отражены в epic. То, что при ручном onboarding’е всплывает через неделю - здесь видно через час.

Важное уточнение: CLI, не MCP. Claude Code работает в терминале - ему доступны все CLI-инструменты, которые установлены на машине. `gh` для GitHub, `jira` для Jira, и так далее. Никакой специальной интеграции - только залогиниться один раз.

### Следующий уровень: чаты и встречи

Если политики компании позволяют - логичное расширение: экспорт чатов и транскриптов встреч. Решения часто принимаются в разговорах, а не в тикетах. “Почему мы выбрали подход X?” - ответ обычно в чате за 3 месяца до того, а не в description тикета.

Это общая проблема onboarding: формальная документация (тикеты, wiki) отражает лишь часть контекста. Остальное - в переписках, созвонах, неформальных решениях. AI может помочь синтезировать и этот слой - если дать ему доступ и если compliance это допускает.

## Just-in-time learning

Классический onboarding - “прочитай всё, потом начни работать”. 50-страничная wiki, README на 20 экранов, welcome pack с ссылками на 30 документов.

Проблема: 80% прочитанного забудется к моменту, когда понадобится. Потому что контекста ещё нет - ты читаешь про deployment pipeline, не понимая зачем он устроен именно так.

Claude Code позволяет другой подход - just-in-time learning. Не “прочитай всё заранее”, а “спроси когда понадобится”. Столкнулся с незнакомым модулем - спросил. Нужно понять deployment flow - спросил. Каждый ответ приходит в контексте реальной задачи, поэтому запоминается.

Документацию всё равно стоит читать - но не всю сразу. CLAUDE.md + 15 минут в Plan Mode дают базу, остальное - по мере необходимости.

## CLAUDE.md как документ для новичка

В главе 3 мы говорили про CLAUDE.md как конфигурацию для Claude. Но есть побочный эффект: это ещё и onboarding document для человека.

Хороший CLAUDE.md содержит: - Как собрать проект - Как запустить тесты - Архитектурные решения (“почему Zustand, а не Redux”) - Gotchas (“pnpm, не npm”) - Куда не лезть

Новый разработчик читает CLAUDE.md - и сразу понимает базу. Набирает `claude` - и может задавать вопросы в контексте всех этих правил. Claude уже знает как проект устроен, какие конвенции, какие подводные камни.

Подход @.claude на PR (глава 3) работает и здесь: Claude наступил на грабли → правило в CLAUDE.md → ни Claude, ни следующий новичок больше не наступят. CLAUDE.md растёт через реальный опыт, а не через попытку описать всё заранее.

## Plan Mode для первого погружения

Shift+Tab → Plan Mode. Claude читает файлы, отвечает на вопросы, НЕ меняет код. Безопасный режим для исследования.

Первый промпт при погружении:

```
Объясни архитектуру этого проекта. Какие основные модули,
как данные текут от входа к выходу, какие ключевые абстракции?
```

Затем:

```
Какие самые изменяемые файлы за последний месяц?
Где горячие точки?
```

И:

```
Какие тесты есть? Покрытие? Что не покрыто?
```

За 15-20 минут в Plan Mode получаешь картину, на которую раньше уходил день чтения кода. А дальше - задаёшь точечные вопросы по мере работы.

## Ограничения и workarounds

Большой epic (40+ тикетов, десятки PR) может не вместиться в один проход - контекст Claude конечен. Практические решения:

- **Несколько проходов.** Сначала структура epic (тикеты + статусы), потом deep dive в конкретные области. Используй /clear между проходами
- **Subagents.** Каждый subagent читает свой блок тикетов, возвращает summary. Основной контекст остаётся чистым (глава 7)
- **Артефакты.** Попроси Claude сохранять промежуточные результаты в файл. Следующая сессия читает файл вместо повторного обхода тикетов

Не пытайся скормить всё за раз. Итеративный подход - быстрее и точнее.

## 5 файлов для DX-инфраструктуры

Минимальный набор markdown-файлов, который делает проект AI-ready и human-friendly:

| Файл | Для кого | Что содержит |
|---|---|---|
| README.md | Человек | Features, установка, примеры |
| CLAUDE.md | AI + человек | Build commands, правила, gotchas |
| ARCHITECTURE.md | Оба | System design, ключевые решения |
| TASKS.md | Оба | Текущий спринт, приоритеты |
| CHANGELOG.md | Оба | Что изменилось и когда |

Пять файлов. Markdown. В git. Обновляются как часть workflow (skills могут делать это автоматически). И сразу - и новый разработчик, и Claude, и ты через полгода можешь быстро погрузиться.

---

# Глава 10: Документация и Knowledge Management

> **TL;DR:** Markdown в git + Claude Code = документация, которую AI может читать и обновлять. Frontmatter для быстрого поиска, authorship tracking для доверия, filesystem > RAG на текущих масштабах.

---

20 минут. Столько я потратил на поиск решения, которое сам записал. Где-то в заметках было, как обойти баг с hot reload в Mantine 8. Точно помню, что записывал. Но в 400+ файлах - не найти.

Это типичная проблема knowledge management: знания есть, но доступа нет. И стандартный ответ - “нам нужен RAG, vector search, embeddings”. Но на практике проблема решается проще.

## Почему markdown

Markdown кажется примитивным. Нет базы данных, нет API, нет schema. Но для AI-first workflow это преимущество:

**Для Claude Code:** - Читается мгновенно (нет парсинга, нет конвертации) - Весь файл - полезная информация (нет UI-мусора) - Структура через заголовки = навигация - Frontmatter = structured metadata

**Для человека:** - Работает в любом редакторе (VS Code, Obsidian, vim, GitHub) - Версионируется в git (diff, blame, history) - Нет vendor lock-in

**Для команды:** - Pull requests на документацию = те же процессы что для кода - Grep по всей базе знаний = одна команда

Notion, Confluence, Google Docs - хорошие инструменты. Но для AI-интеграции markdown в git эффективнее: Claude Code работает с файловой системой напрямую, без API-прослойки.

## Frontmatter caching

Самый высокий leverage для документации - frontmatter с description.

```
---
description: "Архитектура платёжного модуля. Stripe webhooks, idempotency keys,
retry policy (3 попытки, exponential backoff). Решение: отдельная очередь для failed
        payments."
tags: [architecture, payments, stripe, decisions]
---
```

Description - кеш содержимого файла. Ключевые факты, решения, числа - всё что нужно для поиска. Одна строка.

Зачем: grep по description находит релевантные файлы без их открытия. Когда 100+ файлов, разница между “открыть и прочитать каждый” и “grep по frontmatter” - минуты vs секунды.

В Claude Code это ещё мощнее: skill `/search` сначала ищет по frontmatter, и только потом - full-text. Первый проход отсеивает 90% нерелевантных файлов.

Тот баг с Mantine 8? После добавления frontmatter ко всем файлам - нашёл бы за секунды: `grep "Mantine.*hot reload" data/ -r`.

## Authorship tracking

Когда AI генерирует документацию, возникает вопрос: это мнение автора или галлюцинация модели?

Я столкнулся с этим на практике: перечитывал свои заметки и не мог понять - я так думаю, или это Claude написал в прошлой сессии? Голос был похож, содержание правдоподобно, но уверенности нет.

Решение - простое соглашение:

| Маркер | Значение |
|---|---|
| Нет подписи | Написано человеком (дефолт) |
| `> Автор: Claude` | AI-генерация |
| `> Author: [name] + Claude` | Совместная работа |
| `[Author: ...]` | Голос человека внутри AI-документа |

Принцип: **нет подписи = написано человеком**. AI всегда подписывает свою работу. В CLAUDE.md это одно правило - и Claude следует ему автоматически.

## Структура документации для AI

Минимальный набор из главы 9 (5 файлов: README, CLAUDE.md, ARCHITECTURE, TASKS, CHANGELOG) - основа. Но документация проекта шире: ADR, API reference, runbooks, onboarding guides.

Принцип: markdown в git, структура через папки, навигация через CLAUDE.md.

```
docs/
├─ architecture/   # ADR и system design
├─ api/            # API reference
├─ guides/         # How-to для команды
├─ runbooks/       # Операционные процедуры
'- decisions/      # Архитектурные решения из PR
```

Claude Code знает структуру через CLAUDE.md. Skills автоматизируют рутину: - Skill для генерации API docs из кода - Skill для извлечения решений из PR - Skill для проверки актуальности документации

Файлы, которые и человек, и AI могут навигировать. PR на документацию = те же процессы что для кода.

## Filesystem > RAG

Интуитивно кажется: много документов = нужен vector search, embeddings, RAG pipeline.

На практике для большинства проектов это overkill.

| Параметр | Filesystem + grep | RAG pipeline |
|---|---|---|
| Setup | 0 минут | Часы-дни |
| Поддержка | git pull | Reindex при изменениях |
| Точность | 100% (exact match) | Зависит от реализации (semantic) |
| Cost | $0 | Embedding API + vector DB |
| Debugging | grep -n | “Почему не нашло?” |

RAG нужен когда: - 10K+ документов (grep становится медленным) - Семантический поиск критичен (“найди похожие решения”) - Мультимодальный контент (изображения, аудио)

Для типичного проекта - frontmatter + grep + Claude Code хватает с запасом. Не строй инфраструктуру заранее. Переходи на RAG когда filesystem станет узким местом.

## Claude Code для документации

**Генерация из кода:**

```
Прочитай src/api/ и сгенерируй API reference в docs/api.md.
```

Claude читает код напрямую - не нужно копировать в промпт.

**Синхронизация:**

```
Сравни docs/architecture.md с текущим кодом. Что устарело?
```

Документация устаревает. Код - нет. Claude проверяет соответствие.

**Knowledge extraction:**

```
Прочитай все PR за месяц. Выдели архитектурные решения. Добавь в docs/decisions.md.
```

Решения принимаются в PR-ах, чатах, встречах - и теряются. Claude собирает их в одном месте.

---

# Глава 11: Code Review, Quality и Safety

> **TL;DR:** Verification на каждом этапе важнее выбора инструмента. Hooks + Writer/Reviewer pattern + тесты = 2-3x quality без ручного overhead.

---

Телеметрия Faros AI показала: с AI-инструментами PR size вырос на 154%, а время code review - на 91% (глава 8). Разработчики генерируют больше кода, но ревьюерам приходится проверять в 2.5 раза больше. Это не решение - это перекладывание нагрузки.

AI code review инструменты (CodeRabbit, Qodo, Graphite Diamond) обнаруживают больше реальных runtime-багов, чем традиционные static analyzers. Но они работают на уровне PR. Claude Code может работать раньше - на уровне написания кода, до PR.

## Writer/Reviewer pattern

Самый простой и мощный паттерн для quality:

**Сессия A (Writer):** пишет код, решает задачу.

**Сессия B (Reviewer):** получает diff, ревьюит с fresh context.

```
claude -p "Review the changes in the current git diff.
Check for: bugs, edge cases, security issues, performance.
Format: list of issues with severity and line references."
```

Почему работает: Сессия A несёт accumulated bias - Claude “влюбляется” в своё решение. Сессия B - fresh eyes, чистый контекст.

Для критичных изменений:

```
claude -p "You are a security reviewer. Analyze git diff for:
- Injection vulnerabilities (SQL, XSS, command)
- Authentication/authorization flaws
- Secrets in code
Only report confirmed issues with specific line references."
```

## @claude в GitHub PR

Claude Code Action интегрирует Claude Code в GitHub PR workflow:

1. Упоминаешь @claude в комментарии PR
2. Claude анализирует diff, контекст, CLAUDE.md проекта
3. Оставляет review comments или предлагает фиксы
4. Может открыть PR с исправлениями

```yaml
# .github/workflows/claude-review.yml
on:
  pull_request:
  issue_comment:
    types: [created]

jobs:
  claude-review:
    if: contains(github.event.comment.body, '@claude')
    runs-on: ubuntu-latest
    steps:
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
```

CLAUDE.md определяет стандарты review - написал правила один раз, каждый PR проверяется по ним.

## Verification = #1 practice

Boris Cherny:

> “Give Claude a way to verify its work. If Claude has that feedback loop, it will 2-3x the quality of the final result.”

Три уровня verification:

**Уровень 1: Hooks (автоматические проверки)**

PostToolUse после каждого изменения: - prettier проверяет стиль - tsc проверяет типы - feedback мгновенный

Claude редактирует файл → проверки срабатывают → если что-то сломано, Claude узнаёт сразу.

**Уровень 2: Тесты**

```
Напиши функцию X. Сначала напиши тест, потом реализацию. Запусти. Если падает -
исправь.
```

TDD с Claude - не dogma, а pragmatism. Тест = verification layer для самопроверки.

**Уровень 3: Human review**

```
Покажи итоговые изменения перед коммитом. Объясни каждое в одном предложении.
```

До человека доходит уже проверенный код. Меньше nit-picking, больше архитектурных вопросов.

## Claude Code Security

Anthropic [нашла 500+ zero-day уязвимостей] используя Claude для аудита open-source кода. Не pattern matching - reasoning о поведении.

```
Review this authentication flow for security issues.
Consider: token handling, session management, CSRF, input validation.
Focus on logic flaws, not style.
```

В отличие от SAST (Snyk, SonarQube), Claude понимает контекст: “этот input от пользователя используется в SQL без параметризации” - не просто “SQL string concatenation”.

Ограничения: Claude может пропустить уязвимость (false negative) или указать на несуществующую (false positive). AI security review дополняет SAST и ручной аудит, не заменяет их. Для критичного кода - все три слоя.

## Non-negotiables

Пять правил без исключений. Не enterprise platform за $50K - один hook + один config + 10 минут:

**1. Секреты не попадают в модель.** API-ключи через переменные окружения ( `${DATABASE_URL}` ), не через промпт.

**2. Human review обязателен.** AI дополняет, не заменяет. Финальное решение - человеческое.

**3. Чекпойнты по умолчанию.** Esc+Esc откатывает любое действие. Если не можешь откатить - не давай делать.

**4. Опасные команды заблокированы.** PreToolUse hook с exit code 2. Не “договорились”, а “невозможно”.

**5. PR, не прямые коммиты.** Branch → PR → review → merge. Без shortcuts.

Настраивается за 10 минут. Работает без напоминаний. Bottom-Up DevEx в действии: один инженер, один hook, один конфиг.

## Чего избегать

**“Claude написал, значит правильно.”** Trust-then-verify gap. Claude уверенно пишет неправильный код. Без verification - баги в production.

**AI-review вместо human review.** AI ловит паттерны. Человек ловит “зачем”. “Код работает, но решает не ту проблему” - человеческий review.

**Больше кода = больше прогресс.** Кратный рост PR → кратный рост review time → рост багов (глава 8). Больше кода без улучшения процессов = больше проблем.

---

# Глава 12: Claude Code в команде

> **TL;DR:** Shared CLAUDE.md + hooks + skills в git превращают индивидуальный инструмент в командные стандарты. Начинай с champion, не с массовой раздачи лицензий.

---

Представь: новый инженер за час погружается в 4-месячный проект. HTML-отчёт с pipeline, статусами, навигацией по всем тикетам и PR. “Как ты это сделал?” - закономерный вопрос от тимлида.

Честный ответ: индивидуальный Claude Code - полезно. Claude Code на уровне команды - transformative. Но есть подвох.

Ещё более честный ответ: у меня пока нет команды, которая использует Claude Code по стандартам из этой книги. То, что описано ниже - adoption playbook, который я планирую внедрить на full-time DX роли. Часть проверена на практике (shared CLAUDE.md, hooks). Часть - best practices из enterprise кейсов. Разделяю их явно.

## Shared CLAUDE.md (проверено)

CLAUDE.md в корне репозитория, закоммиченный в git - coding standards для AI.

```
# Project Standards

## Build & Test
- `pnpm install` (NOT npm)
- `pnpm test` before any PR

## Architecture
- State: Zustand (NOT Redux)
- Styling: Tailwind (NOT CSS modules)

## Gotchas
- NEVER import from @internal/legacy - deprecated
- Auth tokens refresh every 15 min - handle expiry

## PR Standards
- Max 400 lines changed
- Must have tests for new logic
```

Каждое правило - из реальной ошибки. CLAUDE.md растёт органически.

### @.claude на PR (проверено Boris Cherny)

Claude делает ошибку → ревьюер добавляет правило через @.claude → автоматический commit в CLAUDE.md → ни Claude, ни следующий разработчик не повторят.

CLAUDE.md как institutional memory.

## Shared Skills (проверено)

Skills в `.claude/skills/` коммитятся - команда получает одинаковые workflows:

```
.claude/skills/
  fix-issue/SKILL.md       # /fix-issue 1234
  create-pr/SKILL.md       # /create-pr
  deploy-staging/SKILL.md  # /deploy-staging
  onboard/SKILL.md         # /onboard
```

Новичок набирает `/onboard` - структурированное погружение без отвлечения коллег.

## Shared Hooks (проверено)

Hooks в `.claude/settings.json` - enforcement, не просьба:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "npx prettier --write $FILE_PATH && npx eslint --fix $FILE_PATH"
      }]
    }],
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "bash .claude/hooks/block-dangerous.sh"
      }]
    }]
  }
}
```

Formatting автоматически. Push в main заблокирован. Один стандарт для всех.

## GitHub Actions (enterprise)

Claude Code Action (глава 11) на уровне команды: auto-review каждого PR по правилам из CLAUDE.md, @claude в комментариях для ad-hoc запросов. Один workflow в `.github/workflows/` - и каждый PR проходит через те же стандарты.

## Permission Templates (проверено)

Встроенная система разрешений в `settings.json` - не enterprise-фича, доступна всем:

```
{
  "permissions": {
    "allow": ["Read", "Glob", "Grep", "Edit", "Write", "Bash(pnpm test *)",
        "Bash(pnpm lint *)"],
    "deny": ["Bash(rm -rf *)", "Bash(git push --force *)", "Bash(DROP TABLE *)"]
  }
}
```

`allow` - без переспроса (безопасные инструменты и команды). `deny` - заблокировано полностью. В `.claude/settings.json` проекта - работает для всей команды. Подробности в главе 5.

## Adoption Playbook

### 1. Champion (неделя 1)

Один человек: CLAUDE.md, 2-3 hooks, использует для своих задач. Это роль DX-Champion из Bottom-Up DevEx фреймворка (глава 8) - инженер, который фиксирует friction points и решает их снизу. Показывает результаты. Не “давайте все попробуем”, а “смотрите, я сделал X за 30 минут вместо 3 часов.”

### 2. Expand (недели 2-3)

2-3 skills для team workflows. Коммитит в репозиторий. Ещё 1-2 человека начинают использовать.

### 3. Enforce (недели 3-4)

Hooks для автоматических проверок. GitHub Actions с claude-code-action. Стандарты для всех.

### 4. Measure (неделя 5+)

Baseline survey (глава 13). Метрики: время review, размер PR, onboarding time. Через 4-6 недель - повтор.

### 5. Scale (по мере необходимости)

MCP интеграции. Shared plugins. Agent Teams. Каждый уровень - когда предыдущий стал тесен.

## Когда Claude Code НЕ нужен

Не всем и не всегда:

**Опытный dev в знакомом коде.** METR показал замедление на 19%. Overhead review > benefit. Если сеньор быстрее делает руками - пусть делает руками.

**Команда без базовых процессов.** Нет CI/CD, нет code review, нет тестов? AI усилит хаос. Сначала процессы, потом инструменты.

**Compliance-sensitive контексты.** Если код не может покидать периметр организации - нужен self-hosted (Claude for Enterprise), не обычный Claude Code.

## Anti-patterns

**“Всем раздали лицензии, разберётесь.”** Не работает. PR +154%, review +91%, velocity flat (глава 8).

**Kitchen sink CLAUDE.md.** >150 строк = Claude игнорирует. Фокус: build commands, critical gotchas.

**Trust-then-verify gap.** “Claude написал - наверное правильно.” Без hooks, без тестов = баги в production.

## Enterprise данные

По данным Anthropic customer case studies:

| Компания | Масштаб | Результат |
|---|---|---|
| Spotify | Enterprise-клиент | 650+ AI-changes/month |
| TELUS | 57,000 employees | 30% faster PR turnaround |
| Behavox | Сотни разработчиков | “Go-to pair programmer” |

Общий паттерн: компании с результатами начинали с champions и стандартов, не с массового rollout.

## Claude Code по ролям

Разные роли - разные задачи, разные capabilities, разные метрики.

### DX Engineer

Фокус: снижение friction для всей команды.

| Задача | Что использовать | Метрика |
|---|---|---|
| Ускорить onboarding | CLAUDE.md + `/onboard` skill | Time to first commit |
| Стандартизировать workflows | Shared skills в `.claude/skills/` | % следующих стандартам |
| Автоматизировать проверки | Hooks + GitHub Actions | Defect escape rate |
| Измерить DX | Survey + system metrics (гл.13) | DX Core 4 scores |
| Снизить cognitive load | Permission templates + CLAUDE.md gotchas | Survey: “легко начать задачу?” |

DX Engineer - идеальный champion для внедрения. Он и так решает эти задачи, Claude Code просто кодифицирует решения.

### Tech Lead

Фокус: качество и скорость delivery.

| Задача | Что использовать | Метрика |
|---|---|---|
| Ускорить review | Claude Code Action + verification loops | Review turnaround |
| Уменьшить PR size | CLAUDE.md rules (“max 400 lines”) + hooks | Average PR size |
| Улучшить quality | Writer/Reviewer pattern (гл.11) | Bug rate per release |
| Масштабировать стандарты | Shared CLAUDE.md + @.claude на PR | Consistency across team |
| Onboarding новичков | `/onboard` skill + Plan Mode | Days to first useful PR |

Tech Lead получает value от Claude Code Action в первую очередь - auto-review каждого PR по правилам из CLAUDE.md.

### Platform Engineer

Фокус: инфраструктура и автоматизация.

| Задача | Что использовать | Метрика |
|---|---|---|
| Связать системы | MCP servers (GitHub, Jira, etc.) | Manual copy-paste ops |
| Автоматизировать deploy | Skills + non-interactive mode | Deploy frequency |
| Мониторить quality | Hooks + CI integration | Build success rate |
| Документировать платформу | CLAUDE.md + markdown generation | Docs freshness |
| Batch-обработка | Fan-out pattern (гл.7) | Migration time |

Platform Engineer строит инфраструктуру, на которой работают DX Engineer и Tech Lead. MCP и non-interactive mode - его главные инструменты.

---

# Глава 13: Как измерить DX-улучшения

> **TL;DR:** “Стало быстрее” - не метрика. Baseline survey + system metrics + повтор через 4 недели. DX Core 4, SPACE, DevEx - готовые frameworks, связанные с возможностями Claude Code.

---

“Claude Code ускорил нашу разработку.” Начальник кивает. Потом идёт к своему начальнику. “На сколько ускорил?” - “Ну... ощущение такое.” Конец разговора.

Ощущение - не метрика. METR показал: разработчики ожидали ускорение на +24%, после эксперимента оценивали его на +20%, а реальное измерение показало замедление на -19%. Perception обманчив. А мой зелёный heatmap после месяцев пустоты - correlation, not causation.

Но “onboarding time снизился с 5 дней до 1” - это метрика, с которой можно идти к руководству.

## DX Core 4

GetDX разработала на основе 800+ организаций и 40K+ разработчиков. Четыре измерения:

**Speed** - deployment frequency, lead time, cycle time.

**Effectiveness** - self-reported productivity, % времени на value-add.

**Quality** - defect density, code review effectiveness, reliability.

**Business Impact** - revenue per engineer, feature throughput.

Ключевая цифра: **1 point improvement = 13 минут на разработчика в неделю.** На команде 50 человек - 10+ часов/неделю. На 200 - 45 часов. Это уже разговор на языке бизнеса.

DX Core 4 сочетает surveys (perception) с system metrics (reality). Оба нужны.

## SPACE Framework

Microsoft Research + GitHub + University of Victoria. Пять измерений:

- **Satisfaction** - довольны ли разработчики инструментами и процессами
- **Performance** - качество и impact работы
- **Activity** - объём действий (самая опасная метрика в отрыве от других)
- **Communication** - review turnaround, docs freshness, onboarding time
- **Efficiency & Flow** - deep work, context switches, wait times

Ключевой принцип: **никогда одно измерение**. Activity без Quality = “больше кода с большим количеством багов”. Минимум 3 из 5.

## DevEx (ACM Queue)

Три фактора (Noda, Storey, Forsgren, Greiler - 2023):

**Flow State** - способность погружаться. Время непрерывной работы, прерывания.

**Cognitive Load** - ментальная нагрузка. Количество систем, качество документации.

**Feedback Loops** - скорость обратной связи. Build time, test time, review wait.

Сокращение feedback loops - одно из самых эффективных DX-улучшений. Hooks - прямой инструмент для этого.

## DORA

Самый старый и распространённый фреймворк (с 2014). Четыре классических метрики:

- **Deployment Frequency** - как часто деплоим
- **Lead Time for Changes** - от коммита до продакшена
- **Change Failure Rate** - % деплоев с инцидентами
- **Mean Time to Recovery** - время восстановления после сбоя

DORA - это delivery performance, не developer experience напрямую. Но на практике их часто рассматривают вместе: улучшение DX ведёт к улучшению delivery metrics, и наоборот. Многие менеджеры понимают “улучшить DX” именно как “улучшить DORA metrics”. Поэтому знать их полезно - как минимум для разговора на языке руководства.

В 2024-2025 DORA расширилась далеко за пределы четырёх метрик: добавились вопросы про burnout, documentation, flow, technical debt, AI adoption. Все вопросы открыты под Creative Commons - dora.dev/research/2024/questions.

### 7 DORA Team Archetypes

| Архетип | Приоритет |
|---|---|
| Foundational Challenges | Инфраструктура |
| Code-Centric | Process improvement |
| High Output | Quality gates |
| Fast-Moving | Reliability |
| Well-Balanced | Optimization |
| High-Performing | Innovation |
| Harmonious High-Achievers | Maintenance |

Определи где команда → пойми что улучшать → выбери capabilities Claude Code. “Foundational Challenges” не получит value от Agent Teams - нужны базовые hooks. “High-Performing” не получит value от базового CLAUDE.md - нужны MCP и measurement.

## Mapping: Claude Code → DX метрики

| Capability | DX метрика | Frameworks | Как измерять |
|---|---|---|---|
| Hooks (format, type-check) | Feedback Loops ↓ | DevEx, DORA (lead time) | Время edit → passing check |
| Skills + CLAUDE.md | Cognitive Load ↓ | DevEx, SPACE (efficiency) | Survey: “How easy to get started?” |
| Plan Mode + subagents | Flow State ↑ | DevEx, SPACE (efficiency) | Context switches |
| MCP + GitHub Actions | Speed ↑ | DX Core 4, DORA (deploy freq) | Deployment frequency |
| Auto-memory + sessions | Effectiveness ↑ | DX Core 4, SPACE (performance) | Повторные вопросы/сессию |
| Shared CLAUDE.md + skills | Communication ↑ | SPACE, DX Core 4 | Onboarding time |
| Verification loops | Quality ↑ | DX Core 4, DORA (CFR) | Defect escape rate |
| Sandboxing + permissions | Satisfaction ↑ | SPACE, DevEx | Developer trust in tools |

Подробный маппинг по ролям (DX Engineer, Tech Lead, Platform Engineer) - в главе 12.

## Практический подход

Bottom-Up DevEx: один IC, baseline survey, 4 недели, без бюджета.

### Шаг 0: Friction Log (до внедрения)

Из Bottom-Up DevEx фреймворка (глава 8): прежде чем внедрять инструменты, зафиксируй что именно тормозит. Friction log - структурированный дневник на 2-3 дня:

| Когда | Что делал | Friction | Тип | Severity |
|---|---|---|---|---|
| Пн 10:15 | Настраивал окружение | 20 мин на поиск env-переменных | Docs | High |
| Пн 14:30 | Code review | PR на 800 строк, не понял контекст | Process | Medium |
| Вт 09:00 | Deploy staging | Ручной чеклист из 12 шагов | Automation | High |

Типы: Docs, Tooling, Process, Automation, Communication. Severity: Low/Medium/High.

Friction log показывает где Claude Code даст максимальный эффект. “20 мин на поиск env” → CLAUDE.md gotchas. “Ручной чеклист” → skill. “PR на 800 строк” → hooks + CLAUDE.md rule на max PR size.

### Шаг 1: Baseline survey (день 1)

5-минутный survey для команды (анонимный):

1. Как легко начать работу над новой задачей? (1-5)
2. Как часто feedback на изменения < 10 минут? (1-5)
3. Как часто работаешь без прерываний 2+ часа? (1-5)
4. Насколько доволен инструментами и процессами? (1-5)
5. Сколько времени уходит на “не-кодинг”? (% времени)

Покрывает: Cognitive Load, Feedback Loops, Flow State, Satisfaction, Efficiency. Минимум 3 из 5 SPACE.

Это стартовый набор - достаточный для первого baseline. Можно составить свой, адаптировав под команду. Готовые шаблоны и альтернативные подходы (Atlassian DevEx Survey Play, GetDX, DORA) - в Приложении G.

### Шаг 2: Implement (недели 1-4)

Claude Code по playbook из главы 12: champion → expand → enforce.

### Шаг 3: Measure (неделя 5)

Повтори survey. Сравни.

Дополни system metrics: - Average PR size (git log –stat) - Review turnaround (GitHub API) - Build/test time (CI logs) - Deployment frequency

### Шаг 4: Iterate

Что улучшилось - продолжай. Что не изменилось - разбирайся. Где стало хуже - AI усиливает существующую проблему (глава 8).

## Чего избегать

**Activity как единственная метрика.** Кратный рост PR может сопровождаться кратным ростом review time и ростом багов.

**Сравнение “с AI vs без AI”.** Неправильный вопрос. Правильный: “DX улучшился?” Измеряй результат, не инструмент.

**Measurement без action.** Метрики работают только если за ними следуют изменения. Baseline → implement → measure → iterate. Цикл.

---

# Заключение: Bottom-Up DevEx

> **TL;DR:** Scaffolding важнее модели. 80% автоматизации - детерминированные, 20% - AI. Bottom-Up DevEx: начни от IC, измерь, расширь. Понедельник: CLAUDE.md, 2-3 hooks, 1 skill, baseline survey.

---

13 глав. Hooks, skills, MCP, METR study, DORA paradox, DX Core 4. Много деталей. Но суть укладывается в один подход.

## Bottom-Up DevEx: 5 принципов

Большинство компаний внедряют AI top-down: массовая раздача лицензий без процессов. Результат предсказуем: почти двукратный рост merged PR, delivery metrics flat, review time почти удваивается (глава 8).

Подход этой книги - другой:

1. **Начни от IC, не от менеджмента.** Один инженер, один CLAUDE.md, один hook. Без бюджета, без разрешения, без enterprise agreement
2. **Детерминизм first.** 80% hooks и скрипты, 20% AI. Пример: Trail of Bits (security-компания) заняла 2-е место на DARPA AIxCC - соревновании по автоматическому поиску уязвимостей. Их секрет: преимущественно детерминированный pipeline из static analysis, fuzzing и traditional tools, а LLM - только вспомогательный слой поверх
3. **Измеряй до и после.** Baseline survey до внедрения, не после. “Стало быстрее” - ощущение. “Onboarding 5 дней → 1 день” - метрика
4. **Расти снизу вверх.** CLAUDE.md → skills → hooks → MCP → команда. Каждый следующий слой - когда предыдущий стал тесен
5. **Честно про границы.** AI усиливает, не чинит. Плохие процессы + AI = ещё больший хаос. Сначала процессы

Это не enterprise framework. Это то, что один инженер может начать делать в понедельник.

## Иерархия автоматизации

```
ФУНДАМЕНТ: Ясное мышление
    Понимаешь задачу → формулируешь → AI понимает
    |
АРХИТЕКТУРА: Scaffolding > Model
    CLAUDE.md + hooks + skills важнее версии модели
    |
РЕАЛИЗАЦИЯ: Иерархия автоматизации
    Можно детерминированно?
    ├─ ДА → Код / CLI tool (80%)
    |   '- Обернуть → AI вызывает
    '- НЕТ → AI (20%)

РЕЗУЛЬТАТ: 80% детерминизм, 20% AI
```

PostToolUse hook с `prettier --write` стоит $0 и работает 100% времени. AI-промпт “отформатируй код” стоит токены и работает 95% времени. Выбор очевиден.

## Четыре компонента платформы

| Компонент | Роль | Тип | Глава |
|---|---|---|---|
| **CLAUDE.md** | контракт | advisory | 3 |
| **Skills** | playbooks | workflow | 4 |
| **Hooks** | guardrails | deterministic | 5 |
| **MCP** | руки | integration | 6 |

CLAUDE.md задаёт правила. Skills кодифицируют workflows. Hooks гарантируют compliance. MCP подключает данные. Вместе - платформа. Порядок внедрения - слева направо.

## Что делать с понедельника

1. **Установить Claude Code** (5 минут)

```
curl -fsSL https://claude.ai/install.sh | bash
```

2. **Запустить /init** - пусть Claude проанализирует проект и создаст стартовый CLAUDE.md
3. **Написать 1 skill** для самого частого workflow - тот, который делаешь каждый день и описываешь коллеге каждый месяц
4. **Добавить 2-3 hooks** - auto-format (PostToolUse) + safety (PreToolUse) + notification (Notification)
5. **Закоммитить CLAUDE.md и skills** - чтобы команда получила те же стандарты
6. **Провести baseline survey** - 5 вопросов из главы 13, 5 минут на человека
7. **Через 4 недели** - повторить survey, сравнить

Не всё сразу. Не Agent Teams на второй день. Не enterprise adoption plan. Один CLAUDE.md, один skill, два хука. Результат через неделю.

## Честное закрытие

Не всё описанное в этой книге я применяю на 100%. Agent Teams - пока не в моей практике. MCP использую для двух серверов, не для десяти. Measurement framework из главы 13 - то, что планирую внедрить на full-time DX роли, а не то, что уже работает.

Но CLAUDE.md + hooks + skills - каждый день. Plan Mode для начала работы - каждый день. 80/20 принцип - каждый день. Это base layer, который реально работает.

Эта книга написана теми же инструментами, которые в ней описаны. CLAUDE.md определяет структуру. Hooks проверяют форматирование. Plan Mode для исследований.

## Дальше

Эта книга - snapshot. Claude Code обновляется каждую неделю. DX-практики эволюционируют. Данные накапливаются.

Актуальные материалы, ресурсы и обновления: - **dx-ray.com** - knowledge hub и community по developer experience - **GitHub: DX-Ray** - open source, contributions welcome

---

> Claude Code - не coding assistant. Это DX platform. И как любая платформа, она настолько хороша, насколько хорошо настроены процессы на ней.

# Приложения

## Приложение A: Шаблоны

### Starter CLAUDE.md - React/Next.js

```
# Project Name

## Build & Run

- `pnpm install` to install dependencies
- `pnpm dev` to start development server
- `pnpm build` to build for production
- `pnpm test` to run tests
- `pnpm lint` to lint code

## Architecture

- Framework: Next.js 15 (App Router)
- State: Zustand
- Styling: Tailwind CSS
- API: tRPC
- Database: Prisma + PostgreSQL

## Conventions

- Components in src/components/ (PascalCase.tsx)
- Hooks in src/hooks/ (use-camelCase.ts)
- Tests co-located with source files (\*.test.tsx)

## Gotchas

- Use `pnpm` not `npm` or `yarn`
- Server components by default, "use client" only when needed
- .env.local for development (not .env)
- Database: `pnpm db:push` after schema changes
```

### Starter CLAUDE.md - Python

```
# Project Name

## Build & Run

- `uv sync` to install dependencies
- `uv run pytest` to run tests
- `uv run ruff check .` to lint
- `uv run ruff format .` to format

## Architecture

- Framework: FastAPI
- Database: SQLAlchemy + PostgreSQL
- Tests: pytest + httpx

## Conventions

- Source in src/project_name/
- Tests in tests/ (mirror source structure)
- Type hints required for all public functions

## Gotchas

- Use `uv` not `pip`
- Async handlers: always use `async def`
- Alembic for migrations: `uv run alembic upgrade head`
```

### Starter CLAUDE.md - Go

```
# Project Name

## Build & Run

- `go build ./...` to build
- `go test ./...` to run tests
- `golangci-lint run` to lint

## Architecture

- Standard layout: cmd/, internal/, pkg/
- HTTP: net/http + chi router
- Database: pgx + sqlc

## Conventions

- Interfaces in consumer package
- Errors: wrap with fmt.Errorf("context: %w", err)
- Context: first parameter, always
- Tests: table-driven

## Gotchas

- Go 1.22+ (toolchain directive in go.mod)
- Don't use init() or global state
- sqlc for type-safe queries, not raw SQL
```

### Starter CLAUDE.md - Monorepo

```
# Project Name (Monorepo)

## Build & Run

- `pnpm install` at root
- `pnpm -r build` to build all
- `pnpm --filter @scope/pkg dev` to start specific
- `pnpm test` for all tests

## Structure

- packages/ - shared libraries
- apps/ - deployable applications
- configs/ - shared configs

## Conventions

- Each package has its own CLAUDE.md
- Import via workspace: "@scope/package-name"
- Changes to shared packages: run affected tests

## Gotchas

- Always `pnpm` (workspace protocol)
- Turborepo: `turbo build`
- No circular dependencies
- Changesets for version bumps
```

### hooks.json - Ready-to-use

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(cat | jq -r '.tool_input.file_path // empty') &&
        [ -n \"$FILE\" ] && npx prettier --write \"$FILE\" 2>/dev/null; exit 0"
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "cat | jq -r '.tool_input.command // empty' | grep -qE 'rm
        -rf /|git push.*--force|DROP TABLE' && exit 2 || exit 0"
          }
        ]
      }
    ],
    "Notification": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"Claude Code is
        waiting\" with title \"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

### SKILL.md - /fix-issue

```
---
name: fix-issue
description: Fix a GitHub issue from number to PR
allowed-tools: Read, Edit, Write, Bash, Glob, Grep, Agent
---

1. Read issue #$ARGUMENTS (`gh issue view $ARGUMENTS --json title,body,labels`)
2. Understand requirements and acceptance criteria
3. Find relevant code (Grep, Glob)
4. Implement fix following CLAUDE.md conventions
5. Write or update tests
6. Run tests and linter
7. Create commit and PR linking to issue
```

---

## Приложение B: Чеклист внедрения

### Индивидуальный уровень

- [ ] Claude Code установлен
- [ ] `/init` запущен, CLAUDE.md отредактирован
- [ ] 2-3 hooks (format, safety, notification)
- [ ] 1-2 skills для личных workflows
- [ ] Plan Mode (Shift+Tab) для exploration

### Командный уровень

- [ ] CLAUDE.md закоммичен
- [ ] Hooks в settings.json на уровне проекта
- [ ] 2-3 shared skills
- [ ] MCP для issue tracker
- [ ] GitHub Actions с claude-code-action
- [ ] Permission templates

### Measurement

- [ ] Baseline DX survey (5 вопросов, глава 13)

- [ ] System metrics (PR size, review time, deploy frequency)
- [ ] 4-6 week check-in запланирован
- [ ] Результаты задокументированы

---

## Приложение C: Ссылки и ресурсы

### Исследования

- **METR Study** (2025): рандомизированное контролируемое исследование, -19% для опытных devs - metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study
- **DORA Report 2025**: "AI's primary role is as an amplifier" - dora.dev/research/2025/dora-report
- **GetDX/DX Core 4**: 1 point = 13 минут/неделю, 800+ организаций - getdx.com
- **DevEx (ACM Queue, 2023)**: Noda, Storey, Forsgren, Greiler - Flow, Cognitive Load, Feedback Loops - queue.acm.org/detail.cfm?id=3595878

### Инструменты

- **Claude Code** - AI-ассистент для разработки в терминале
- **Claude Code Action** - GitHub Action для автоматического code review
- **MCP** - Model Context Protocol, стандарт интеграции AI с внешними системами
- **MCP Servers** - коллекция готовых MCP-серверов

### Рекомендуемое чтение

- **"Why Developers Don't Know What Dev Ex Is" (Artem Mukhin, The New Stack)** - bottom-up DX подход
- **"The Bottom-Up Approach to Developer Experience" (Artem Mukhin, 2026)** - 4-этапный фреймворк (Friction Log → DX-Champion → Initiatives → Leadership), теоретическая база (DevEx, CLT, STC)
- **"Frictionless" (Forsgren & Noda)** - developer experience methodology
- **"Developer Experience" (Addy Osmani)** - DX primer, бесплатно онлайн
- **"Beyond Vibe Coding" (Addy Osmani)** - AI-assisted development practices
- **Anthropic Skills docs** - документация по skills

### DX-Ray

- Курированная библиотека материалов по DX и открытое комьюнити

## Приложение D: Bottom-Up DevEx Checklist

4-недельный план для одного инженера, без бюджета.

### Week 0: Baseline

- [ ] **Friction log** (2-3 дня): записывай каждый момент friction - что тормозит, тип (Docs/Tooling/Process/Automation/Communication), severity
- [ ] Установить Claude Code
- [ ] Провести baseline survey (5 вопросов из главы 13)
- [ ] Записать текущие метрики: onboarding time, review turnaround, deploy frequency
- [ ] Определить top-3 friction points из лога → они станут первыми targets для CLAUDE.md, hooks, skills

### Week 1: Foundation

- [ ] `/init` → CLAUDE.md написан и закоммичен
- [ ] Первый hook: auto-format (PostToolUse)
- [ ] Второй hook: safety (PreToolUse - блокировка опасных команд)
- [ ] Notification hook (macOS/Telegram)
- [ ] Попробовать Plan Mode для exploration незнакомого кода

### Week 2: Workflows

- [ ] 1-2 skills для самых частых workflows ( `/fix-issue` , `/deploy` , `/onboard` )
- [ ] Skills закоммичены в `.claude/skills/`
- [ ] MCP подключен если нужен (GitHub, issue tracker)
- [ ] 1-2 коллеги начали использовать

### Week 3: Standards

- [ ] CLAUDE.md обогащён: gotchas, architecture decisions, conventions
- [ ] Hooks в `.claude/settings.json` (для всей команды)
- [ ] GitHub Actions с claude-code-action (если GitHub)
- [ ] Permission templates настроены

### Week 4: Measure

- ☐ Повторный survey (те же 5 вопросов)
- ☐ Сравнить с baseline
- ☐ Собрать system metrics: PR size, review time, deploy frequency
- ☐ Задокументировать: что улучшилось, что нет, следующие шаги

---

## Приложение E: Когда НЕ использовать Claude Code

### Anti-recommendations

**Опытный dev на знакомом коде.** METR: -19%. Если знаешь кодовую базу вдоль и поперёк и задача типовая - руки быстрее. AI добавляет overhead на review и acceptance.

**Без базовых процессов.** Нет CI/CD, нет code review, нет тестов? AI усилит хаос. Сначала процессы, потом инструменты.

**Compliance/security constraints.** Код не может покидать периметр? Нужен Claude for Enterprise (VPC deployment), не обычный Claude Code.

**Простые одноразовые задачи.** “Переименуй переменную” быстрее сделать через IDE rename refactoring, чем объяснять Claude.

**Когда нужен контроль каждой строки.** Performance-critical код, криптография, security-sensitive логика - здесь AI может ввести subtle bugs. Human-written, AI-reviewed.

### Когда AI добавляет overhead

- Review AI-генераций занимает больше времени, чем написать самому
- Claude предлагает >50% решений, которые ты отклоняешь
- Сессии >1 часа без /clear (контекст деградирует)
- Промпт длиннее 3 предложений для простой задачи

### Правило: если сомневаешься

Попробуй обе стороны. Сделай задачу с Claude Code, замерь время. Сделай похожую без. Сравни. Perception обманчив (METR: ожидали +24%, получили -19%).

---

## Приложение F: Дополнительные рецепты hooks

Рецепты из главы 5, вынесенные для reference.

### Context priming при старте сессии

SessionStart hook загружает контекст автоматически:

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/session-context.sh"
          }
        ]
      }
    ]
  }
}
```

`.claude/hooks/session-context.sh`:

```
#!/bin/bash
cat <<EOF
{
  "suppressOutput": false,
  "message": "Active priorities: $(cat docs/PRIORITIES.md 2>/dev/null || echo
        'none')\nRecent changes: $(git log --oneline -5 2>/dev/null || echo 'no
        git')"
}
EOF
```

### TypeScript проверка после редактирования

PostToolUse hook для .ts/.tsx файлов:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/typecheck.sh"
          }
        ]
      }
    ]
  }
}
```

`.claude/hooks/typecheck.sh`:

```
#!/bin/bash
INPUT=$(cat)
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

if [[ "$FILE" == *.ts || "$FILE" == *.tsx ]]; then
  ERRORS=$(npx tsc --noEmit 2>&1 | head -20)
  if [ $? -ne 0 ]; then
    echo "$ERRORS"
    # Exit 0: не блокируем, но показываем ошибки Claude
  fi
fi
```

### Бэкап транскрипта перед compaction

PreCompact hook сохраняет полный транскрипт:

```
#!/bin/bash
BACKUP_DIR=".claude/transcripts"
mkdir -p "$BACKUP_DIR"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
INPUT=$(cat)
echo "$INPUT" > "$BACKUP_DIR/pre-compact-$TIMESTAMP.json"
```

### Telegram push при завершении задачи

Stop hook для уведомлений:

```
#!/bin/bash
if [ -n "$TELEGRAM_BOT_TOKEN" ]; then
  curl -s -X POST "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/sendMessage" \
    -d chat_id="$TELEGRAM_CHAT_ID" \
    -d text="Claude Code завершил задачу" \
    > /dev/null 2>&1
fi
```

---

## Приложение G: DX Survey Frameworks

Обзор готовых фреймворков и шаблонов для измерения developer experience. Пять вопросов из главы 13 - стартовый набор. Здесь - куда двигаться дальше.

### Atlassian DevEx Survey Play (бесплатный)

Готовый playbook с шаблоном. 8 “vital signs”:

1. Sustainable speed to ship
2. Waiting time
3. Execution independence
4. Ways of working
5. External standards
6. Maintenance
7. Onboarding
8. Developer satisfaction

Каждый vital sign оценивается по двум шкалам: **importance** (насколько важно) + **satisfaction** (насколько довольны). Opportunity score = `importance + max(importance - satisfaction, 0)`. Рекомендуют 6-8 vital signs, запускать раз в полгода минимум.

Ссылка: [atlassian.com/team-playbook/plays/devex-survey](atlassian.com/team-playbook/plays/devex-survey)

### GetDX / DX Core 4 (коммерческий)

14-вопросный Developer Experience Index (DXI). Вопросы проприетарные, но покрывают 5 областей: team processes, tooling, focus time, quality practices, overall satisfaction. Бесплатный шаблон (gated за формой): getdx.com/guide/devex-survey-template.

Ключевая цифра: 1 point improvement = 13 минут на разработчика в неделю.

### SPACE (Microsoft Research, 2021)

5 измерений с примерами вопросов:

| Измерение | Пример вопроса |
|---|---|
| Satisfaction | “Насколько вы довольны своей работой за последние недели?” |
| Performance | “Насколько продуктивной была команда?” |
| Activity | “Насколько вовлечена команда в инструменты и процессы?” |
| Communication | “Насколько эффективны коммуникация и knowledge sharing?” |
| Efficiency & Flow | “Удаётся ли завершать задачи без задержек и прерываний?” |

Ключевой принцип: никогда одно измерение. Минимум 3 из 5.

### DevEx (ACM Queue, 2023)

Noda, Storey, Forsgren, Greiler. Три фактора: Flow State, Cognitive Load, Feedback Loops. Не дают готовый шаблон, но дают фреймворк для составления вопросов. Наиболее академически обоснованный подход.

Ссылка: queue.acm.org/detail.cfm?id=3595878

### DORA (с 2014, расширен в 2024)

Самый старый и распространённый. Изначально - 4 метрики delivery performance (deployment frequency, lead time, change failure rate, MTTR). Не про DX напрямую, но часто используется вместе: улучшение DX ведёт к улучшению DORA metrics.

В 2024 расширились до 100+ вопросов по 20+ категориям: burnout, documentation, flow, technical debt, code quality, productivity, AI adoption. Все вопросы открыты под Creative Commons.

Ссылка: dora.dev/research/2024/questions

### Какой выбрать

| Ситуация | Рекомендация |
|---|---|
| Быстрый baseline, нет бюджета | 5 вопросов из главы 13 |
| Нужна методология importance/satisfaction | Atlassian DevEx Survey Play |
| Enterprise, нужны benchmarks | GetDX DX Core 4 |
| Академическая строгость | DevEx (ACM) + SPACE |
| Разговор с менеджментом про delivery | DORA metrics |
| Всё вместе | DX Core 4 (объединяет DORA + SPACE + DevEx) |